

Contents

1	Linear Programming	2
1.1	Geometry of Linear Programming	2
1.2	Standard Form of Linear Programs	4
1.3	Convex Sets and Functions	6
2	The Simplex Method	9
2.1	Basic Feasible Solutions	9
2.2	The Simplex Method	15
2.3	Tableau Implementation	22
2.4	Finding An Initial Basic Feasible Solution	23
2.5	Special Cases	25
3	Duality Theory	27
3.1	The Dual Problem	27
3.2	Duality Theorems	29
3.3	Dual Simplex Method	32
3.4	Sensitivity Analysis	33
4	Network Optimisation	36
4.1	Network Flow Problems	36
4.1.1	Single-Source Shortest Path Problem	38
4.1.2	Maximum Flow Problem	43
4.2	Network Simplex Method	45
4.3	Integrality	48

Linear Programming

Recall that in general, an optimisation problem can be formulated as

$$\begin{aligned} \min_{\mathbf{x} \in X} & f(\mathbf{x}) \\ \text{s.t. } & g_i(\mathbf{x}) = 0 \quad \text{for } i = 1, 2, \dots, p \\ & h_j(\mathbf{x}) \leq 0 \quad \text{for } j = 1, 2, \dots, m, \end{aligned}$$

where f is known as the *objective function*, g_i 's are known as *equality constraints* and h_j 's are known as *inequality constraints*. The set of all $\mathbf{x}$ that satisfy all constraints is called the *feasible set*, where each of such $\mathbf{x}$ is a *feasible solution*. The vector which minimises f is known as the *optimal solution* and is denoted by $\mathbf{x}^*$, with $f(\mathbf{x}^*)$ being the *optimal value*.

The concept of a *linear program* is intuitive to understand: it is simply an optimisation problem whose objective function and constraint functions are all linear. In a 2-dimensional plane, we see that any region bounded by linear functions is a polygon. We will abstract this idea and generalise it for any finite-dimensional space.

1.1 Geometry of Linear Programming

In any Euclidean space $\mathbb{R}^n$, a linear function can be written as

$$f(x_1, x_2, \dots, x_n) = a_0 + \sum_{i=1}^n a_i x_i,$$

where the a_i 's are real coefficients. In matrix notations, this becomes

$$f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} + a_0$$

for some $\mathbf{c} \in \mathbb{R}^n$. Let $f(\mathbf{x}) = a_0 + b$, then we have $\mathbf{c}^T \mathbf{x} = b$. Note that this level set equation gives a linear function in $\mathbb{R}^{n-1}$, because obviously there exists some $\mathbf{p} \in \mathbb{R}^{n-1}$ such that $\mathbf{c}^T \mathbf{x} = c_n x_n + \mathbf{p}^T \mathbf{x}'$, where $\mathbf{x}' = (x_1, x_2, \dots, x_{n-1})$. Therefore,

$$x_n = \frac{b}{c_n} - \frac{1}{c_n} \mathbf{p}^T \mathbf{x}'.$$

Apparently, x_n is a linear function of $\mathbf{x}'$. This means every straight line in $\mathbb{R}^n$ is defined by an equation

$$\mathbf{a}^T \mathbf{x} = b$$

for some $\mathbf{a} \in \mathbb{R}^n$ and $b \in \mathbb{R}$. Take $\mathbf{x}_0 \in \mathbb{R}^n$ which is on this line and let $\mathbf{d} \in \mathbb{R}^n$ be the direction vector of the line, then for every $\mathbf{x}$ with $\mathbf{a}^T \mathbf{x} = b$, we also have

$$\mathbf{x} = \mathbf{x}_0 + \lambda \mathbf{d}$$

for some $\lambda \in \mathbb{R}$. However, this implies that for any such $\mathbf{x}$, we have

$$\mathbf{a}^T(\mathbf{x}_0 + \lambda \mathbf{d}) = \mathbf{a}^T \mathbf{x} = b,$$

but $\mathbf{a}^T \mathbf{x}_0 = b$, so we have to have $\mathbf{a}^T \mathbf{d} = 0$. Therefore, the set

$$A_\perp := \{\mathbf{x} : \mathbf{a}^T \mathbf{x} = b\}$$

in fact is a set of vectors orthogonal to $\mathbf{a}$ in $\mathbb{R}^n$.

Definition 1.1.1 ► Hyperplane

Let $\mathbf{a} \in \mathbb{R}^n$ be a vector. For any $b \in \mathbb{R}$, the set

$$H_b := \{\mathbf{x} : \mathbf{a}^T \mathbf{x} = b\}$$

is said to be a **hyperplane** with normal vector $\mathbf{a}$.

It is easy to see that in $\mathbb{R}^2$, a hyperplane is a straight line perpendicular to $\mathbf{a}$ and in $\mathbb{R}^3$, it is a plane whose normal vector is parallel to $\mathbf{a}$.

Intuitively, a hyperplane partitions the space $\mathbb{R}^n$ into 2 halves. Therefore, we refer to the set

$$\{\mathbf{x} : \mathbf{a}^T \mathbf{x} \leq b\}$$

as a *half-space*. Intuitively, if we have m half-spaces, then their intersection is a polyhedron in the space, i.e., we define the set

$$P := \bigcap_{i=1}^m \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}_i^T \mathbf{x} \leq b_i\}$$

as a *polyhedral set*.

Remark. Note that for P to be a polyhedron, the intersection must be finite. Otherwise, consider the counter example of the bounded set whose boundary is defined by all hyperplanes at a distance d away from a fixed point Q , which is a sphere.

We can let $\mathbf{a}_i^T$ be the i -th row of the matrix $\mathbf{A}$ and define a column vector $\mathbf{b}$ whose i -th entry is b_i , then we can re-write the above intersection using matrix multiplication.

Definition 1.1.2 ► Polyhedron

A **polyhedron** is defined as the set

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \leq \mathbf{b}\}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$.

1.2 Standard Form of Linear Programs

Definition 1.2.1 ► Linear Programming Problem

A **linear programming** (LP) problem is an optimisation problem where the objective function f is linear and the feasible set P is a polyhedron.

Note that each linear constraint corresponds to a half-space, so we can formulate a linear programming problem as

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} & \mathbf{c}^T \mathbf{x} \\ \text{s.t. } & \mathbf{a}_i^T \mathbf{x} \leq b_i \quad \text{for } i = 1, 2, \dots, p \\ & \mathbf{a}_j^T \mathbf{x} = b_j \quad \text{for } i = 1, 2, \dots, m, \\ & \mathbf{a}_k^T \mathbf{x} \neq b_k \quad \text{for } i = 1, 2, \dots, q. \end{aligned}$$

where $\mathbf{c} \in \mathbb{R}^n$ is called the *cost* or *profit* vector, $\mathbf{a}_i^T \mathbf{x}$, $\mathbf{a}_j^T \mathbf{x}$ and $\mathbf{a}_k^T \mathbf{x}$ are called the *constraints* and $\mathbf{x}$ is known as *decision variables*. Note that the number of constraints must be finite, or else we may have a feasible set which is not a polyhedron.

Given any linear program, it may present one of the following possibilities:

1. The program has a unique optimal solution.
2. The program has infinitely many optimal solution (but still a unique optimal value).
3. The program has no optimal solution.

4. The program has no feasible solution.

The first 2 cases are trivial. Suppose a linear program with objective function f has no optimal solution, then it means for every feasible $\mathbf{x}$, we can find a different feasible $\mathbf{x}'$ such that $f(\mathbf{x}') < f(\mathbf{x})$.

Suppose a linear program with objective function f has no feasible solution, then this means that the feasible set is empty. In this case, we define the optimal value to be ∞ . The reasoning is as follows.

Suppose f and g are objective functions of 2 optimisation problems with feasible set S_1 and S_2 respectively such that $S_1 \subseteq S_2$. Clearly, $\min f(\mathbf{x}) \geq \min g(\mathbf{x})$. Note that if $S_1 = \emptyset$, then for every S_2 , we have the above inequality, which means that

$$\min_{\mathbf{x} \in S_1} f(\mathbf{x}) \geq y$$

for all $y \in \mathbb{R}$. Therefore, $\min_{\mathbf{x} \in S_1} f(\mathbf{x}) = \infty$.

For each inequality constraint in the form of $\mathbf{a}_i^T \mathbf{x} \leq b_i$, we can introduce a *slack variable* $s_i \geq 0$ such that $\mathbf{a}_i^T \mathbf{x} + s_i = b_i$. For each constraint on x_i in the form of $x_i \leq 0$, we can replace every occurrence of x_i by $-x_i^- = x_i$ such that $x_i^- \geq 0$. For each free variable x_j , we can express it as

$$x_j = x_j^+ - x_j^- \quad \text{for some } x_j^+, x_j^- \geq 0.$$

For instance, we can take $x_j^+ = 0$ and $x_j^- > 0$ whenever $x_j < 0$ and vice versa for $x_j > 0$. Note that this correspondence is not unique.

After the above transformations, we see that every constraint is equivalent to either an equality constraint $\mathbf{a}_i^T \mathbf{x} + s_i = b_i$ or an inequality constraint in the form of $x_j \geq 0$. Therefore, we define the following as the *standard form* of a linear program:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \mathbf{Ax} = \mathbf{b} \\ & x_j \geq 0, \quad \text{for } j = 1, 2, \dots, n. \end{aligned}$$

Note that here, $\mathbf{A}$ is an $m \times n$ matrix where the i -th row in $\mathbf{A}$ contains the coefficients of the i -th constraint.

One should realise that a linear program in the standard form can be more easily solved by using linear algebra to find the optimal solution. Note that not every optimisation problem is given in the standard form. Fortunately, we can always convert a linear program into the standard form.

1.3 Convex Sets and Functions

Intuitively, we describe two types of shapes in natural languages: the shapes which, if you choose any of its edges, lies in the same side of that edge, and the shapes which span across both sides from some chosen edge of its.

Graphically, this means that some shapes are “convex” to all directions, where as some other shapes are “concave”. We shall define this rigorously as follows:

Definition 1.3.1 ► Convex Set

A set $D \subseteq \mathbb{R}^n$ is said to be **convex** if for all $\mathbf{x}, \mathbf{y} \in D$ and for all $\lambda \in [0, 1]$,

$$\lambda \mathbf{x} + (1 - \lambda) \mathbf{y} \in D.$$

Analogously, we might want to say that a function is convex if, for any 2 points on its graph, the line segment joining the 2 points “lies within” the graph of the function. We can define convexity over functions as follows:

Definition 1.3.2 ► Convex Function

A function $f : D \rightarrow \mathbb{R}^n$ is said to be **convex** if for all $\mathbf{x}, \mathbf{y} \in D$ and for all $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

Definition 1.3.3 ► Concave Function

A function $f : D \rightarrow \mathbb{R}^n$ is said to be **concave** if for all $\mathbf{x}, \mathbf{y} \in D$ and for all $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \geq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

From another perspective, we can see that for any convex function f , the tangent plane to the graph of f at any point will lie below the graph.

Remark. A function which is not convex must be concave. However, a function which is convex may not be non-concave (consider $f(x) = x$).

Therefore, it is easy to see that all functions in the form of $f(\mathbf{x}) = d + \mathbf{c}^T \mathbf{x}$ are both convex and concave. Such functions are said to be *affine* functions. Equivalently, this means that all affine functions are neither strictly convex nor strictly concave.

In the above definitions, the expression $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$ for $\lambda \in [0, 1]$ is known as a *convex combination*. This notion can be generalised for any finite number of terms.

Proposition 1.3.4 ► Generalised Convex Combination

Let $k \in \mathbb{N}^+$ and let $f : S \rightarrow \mathbb{R}$ be a convex function on the convex set $S \subseteq \mathbb{R}^n$ and let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k \in S$, then

$$f\left(\sum_{i=1}^k \lambda_i \mathbf{x}_i\right) \leq \sum_{i=1}^k \lambda_i f(\mathbf{x}_i),$$

where $\sum_{i=1}^k \lambda_i = 1$ and $\lambda_i \geq 0$ for $i = 1, 2, \dots, k$.

Using the idea of convex combinations, we can define the notion of a *convex hull*.

Definition 1.3.5 ► Convex Hull

The **convex hull** of $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ is defined as the set of all convex combinations of the vectors, denoted by

$$\text{conv}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n) := \left\{ \mathbf{x} \in \mathbb{R}^n : \mathbf{x} = \sum_{i=1}^n \lambda_i \mathbf{x}_i, \lambda_i \in [0, 1], \sum_{i=1}^n \lambda_i = 1 \right\}.$$

Note that $\text{conv}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$ is the smallest convex set containing all of $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$.

Now we consider the following proposition:

Proposition 1.3.6 ► Maximum of Convex Functions Is Convex

Let $f_1, f_2, \dots, f_m : \mathbb{R}^n \rightarrow \mathbb{R}$ be convex functions, then the function

$$f(\mathbf{x}) := \max_{i=1,2,\dots,m} f_i(\mathbf{x})$$

is convex.

Proof. Take any $\mathbf{x} \neq \mathbf{y} \in \mathbb{R}^n$ and consider $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$ for some $\lambda \in [0, 1]$. Note that for each of the f_i 's, we have

$$f_i(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f_i(\mathbf{x}) + (1 - \lambda) f_i(\mathbf{y}),$$

and so

$$\max_{i=1,2,\dots,m} f_i(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \max_{i=1,2,\dots,m} [\lambda f_i(\mathbf{x}) + (1 - \lambda) f_i(\mathbf{y})].$$

Therefore,

$$\begin{aligned} f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) &= \max_{i=1,2,\dots,m} f_i(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \\ &\leq \max_{i=1,2,\dots,m} [\lambda f_i(\mathbf{x}) + (1 - \lambda) f_i(\mathbf{y})] \\ &= \lambda \max_{i=1,2,\dots,m} f_i(\mathbf{x}) + (1 - \lambda) \max_{i=1,2,\dots,m} f_i(\mathbf{y}) \\ &= \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \end{aligned}$$

□

An immediate corollary of Proposition 1.3.6 allows us to define a piece-wise convex affine function.

Corollary 1.3.7 ► Piece-wise Affine Functions Are Convex

The piece-wise affine function

$$f(\mathbf{x}) = \max_{i=1,2,\dots,n} (\mathbf{c}_i^T \mathbf{x} + d_i)$$

is convex.

The Simplex Method

2.1 Basic Feasible Solutions

Recall that given any linear program, its feasible set is a polyhedron P . Note that in $\mathbb{R}^n$, the smallest possible number of hyperplanes intersecting at a point is n . Intuitively, any polyhedron can be completely defined by all of such “corner points” of itself. Here we provide three equivalent definitions.

Definition 2.1.1 ▶ Extreme Point

Let P be a polyhedron, a point $\mathbf{x}^* \in P$ is said to be an **extreme point** if whenever there are $\mathbf{y}, \mathbf{z} \in P$ with $\mathbf{x}^* = \lambda \mathbf{y} + (1 - \lambda) \mathbf{z} = \mathbf{x}^*$ for some $\lambda \in (0, 1)$, we have $\mathbf{y} = \mathbf{z} = \mathbf{x}^*$.

We can interpret the definition as follows: suppose $\mathbf{x}^*$ is not a corner point, then there are 2 possibilities. If $\mathbf{x}^*$ is an internal point, then there is some $\delta > 0$ such that the neighbourhood $V_\delta(\mathbf{x}^*) \subseteq P$. Therefore, we can always find $\mathbf{y}, \mathbf{z} \in V_\delta(\mathbf{x}^*)$ with $\mathbf{y} \neq \mathbf{z} \neq \mathbf{x}^*$ such that $\mathbf{x}^* \in (\mathbf{y}, \mathbf{z})$. Otherwise, $\mathbf{x}^*$ is on the boundary, i.e.,

$$\mathbf{x}^* \in H := \{\mathbf{x} : \mathbf{a}^T \mathbf{x} \leq b\}.$$

Clearly, we can also find $\mathbf{y}, \mathbf{z} \in V_\delta(\mathbf{x}^*) \cap H$ with $\mathbf{y} \neq \mathbf{z} \neq \mathbf{x}^*$ such that $\mathbf{x}^* \in (\mathbf{y}, \mathbf{z})$.

Alternatively, we consider a hyperplane defined by $\mathbf{c}^T \mathbf{x} = b$ such that $\mathbf{c}$ is not orthogonal to any of the boundaries of the polyhedron P . Clearly, $\mathbf{c}$ is the gradient vector of the affine function

$$f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}.$$

Therefore, by translating the hyperplane in the direction of $\mathbf{c}$, we are able to maximise b . Note that this hyperplane is not parallel to any boundary of the polyhedron, so when b reaches the maximum, the hyperplane will have a unique intersection with the polyhedron, which must be a corner point.

Definition 2.1.2 ▶ Vertex

Let P be a polyhedron, a point $\mathbf{x}^* \in P$ is said to be a **vertex** if there exists some $\mathbf{c}$ such that $\mathbf{c}^T \mathbf{x}^* > \mathbf{c}^T \mathbf{y}$ for all $\mathbf{y} \in P - \{\mathbf{x}^*\}$.

Note that here we require the inequality to be strict, because otherwise $\mathbf{x}^*$ and $\mathbf{y}$ may both be internal points of some boundary hyperplane of P .

Note that any boundary of a polyhedron P is uniquely determined by a constraint $\mathbf{a}_i^T \mathbf{x} \leq b_i$. Let $\mathbf{x}^* \in P$, it is clear that $\mathbf{x}^*$ is “on the boundary” if and only if $\mathbf{a}_i^T \mathbf{x} = b_i$ for some i . In such cases, we say that the corresponding constraint is *active/binding/tight* at $\mathbf{x}^*$.

Geometrically, a corner point of a polyhedron P in $\mathbb{R}^n$ is the intersection of n boundary hyperplanes which are pair-wise non-parallel. Let $\mathbf{a}_i^T \mathbf{x} = b_i$ and $\mathbf{a}_j^T \mathbf{x} = b_j$ be any of the n hyperplanes, then it is clear that $\mathbf{a}_i$ and $\mathbf{a}_j$ must be linearly independent for the two hyperplanes to be non-parallel. This leads to the notion of *basic feasible solutions*.

Definition 2.1.3 ► Basic Feasible Solution

Let $P \subseteq \mathbb{R}^n$ be a polyhedron. $\mathbf{x}^* \in P$ is said to be a **basic feasible solution** if there are n linearly independent constraints which are active at $\mathbf{x}^*$.

We can generalise Definition 2.1.3 to deal with even infeasible points. First we introduce some terminologies.

Definition 2.1.4 ► Rank

Let $P \subseteq \mathbb{R}^n$ be a polyhedron with constraints $\mathbf{a}_i^T \mathbf{x} \leq b_i$ for $i = 1, 2, \dots, m$. For any $\mathbf{x} \in \mathbb{R}^n$, the **rank** of $\mathbf{x}$ is defined as

$$\text{rank}(\mathbf{x}) := \dim(\text{span}\{\mathbf{a}_j : \mathbf{a}_j^T \mathbf{x} = b_j\}).$$

Clearly, $\mathbf{x}^* \in \mathbb{R}^n$ is a basic feasible solution if and only if $\text{rank}(\mathbf{x}^*) = n$ and $\mathbf{x}^* \in P$. Notice that not all $\mathbf{x}$ with rank n is feasible, so we might consider the following definition:

Definition 2.1.5 ► Basic Solution

Let $P \subseteq \mathbb{R}^n$ be a polyhedron. A vector $\mathbf{x} \in \mathbb{R}^n$ is a **basic solution** if $\text{rank}(\mathbf{x}) = n$.

One important thing to note here is that $\text{rank}(\mathbf{x}) = n$ does not necessarily imply that there are exactly n constraints active at $\mathbf{x}$. Intuitively, there can be more than n hyperplanes in $\mathbb{R}^n$ which intersect at a point $\mathbf{x}$. However, if $\text{rank}(\mathbf{x}) = n$, then some hyperplanes are “redundant”, i.e., their normal vectors can be expressed as linear combinations of the gradients of some n linearly independent constraints.

Definition 2.1.6 ► Degeneracy

A basic solution $\mathbf{x} \in \mathbb{R}^n$ is said to be **degenerate** if there are more than n constraints active at $\mathbf{x}$.

Let $P \subseteq \mathbb{R}^n$ be a polyhedron defined by m constraints. Clearly, for each basic feasible solution, we require at least n different constraints to be active. Therefore, the number of distinct basic feasible solutions in P is at most $\binom{m}{n}$. This justifies the fact that any polyhedron in $\mathbb{R}^n$ determined by less than n constraints has no basic feasible solution, and any polyhedron in a finite-dimensional space must have finitely many basic feasible solutions.

Suppose we are given a standard linear program, then we can write its feasible set as the polyhedron

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\},$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$. We will devise a way to compute the basic feasible solutions systematically.

Theorem 2.1.7 ► Basic Solution Characterisation

Let

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

be a polyhedron for some $\mathbf{A} \in \mathbb{R}^{m \times n}$. A vector $\mathbf{x}^* \in \mathbb{R}^n$ is a basic solution if and only if

- $\mathbf{Ax}^* = \mathbf{b}$, and
- There exists an index set $B \subseteq \{1, 2, \dots, n\}$ such that the set

$$\{\mathbf{A}_i : i \in B\}$$

is linearly independent and $x_j^* = 0$ for all $j \notin B$, where

$$\mathbf{A} = [\mathbf{A}_1 \quad \mathbf{A}_2 \quad \dots \quad \mathbf{A}_n].$$

Proof. Write $B = \{B(1), B(2), \dots, B(m)\}$ and define

$$N = \{N(1), N(2), \dots, N(n-m)\} := \{1, 2, \dots, n\} - B.$$

For each $i \in N$, since $x_i^* = 0$, we have $\mathbf{e}_i^T \mathbf{x}^* = 0$. Therefore, the matrix representation for the active constraints is

$$\begin{bmatrix} \mathbf{A} \\ \mathbf{e}_{N(1)}^T \\ \mathbf{e}_{N(2)}^T \\ \vdots \\ \mathbf{e}_{N(n-m)}^T \end{bmatrix} \mathbf{x}^* = \mathbf{0}.$$

Re-arranging the columns, the above matrix can be re-written as

$$\begin{bmatrix} \mathbf{A}_B & \mathbf{A}_N \\ \mathbf{0} & \mathbf{I}_N \end{bmatrix} \bar{\mathbf{x}}^* = \mathbf{0},$$

where $\bar{\mathbf{x}}^*$ is obtained by re-arranging the rows of $\mathbf{x}^*$ accordingly. Note that the columns of $\mathbf{A}_B$ is linearly independent, so $\det(\mathbf{A}_B) \neq 0$. Therefore,

$$\begin{vmatrix} \mathbf{A}_B & \mathbf{A}_N \\ \mathbf{0} & \mathbf{I}_N \end{vmatrix} = \det(\mathbf{A}_B) \det(\mathbf{I}_N) \neq 0,$$

and so the matrix is invertible. Therefore, the rows of the matrix are linearly independent. This means that there are n linearly independent constraints active at $\mathbf{x}^*$. Therefore, $\mathbf{x}^*$ is a basic feasible solution.

Suppose conversely that $\mathbf{x}^*$ is a basic feasible solution, then clearly $\mathbf{A}\mathbf{x}^* = \mathbf{b}$. Since there are m equality constraints, then we must have $(n - m)$ active inequality constraints at $\mathbf{x}^*$, indexed by $N = \{N(1), N(2), \dots, N(n - m)\}$, such that the constraints are linearly independent. Therefore, the matrix

$$\begin{bmatrix} \mathbf{A} \\ \mathbf{e}_{N(1)}^T \\ \mathbf{e}_{N(2)}^T \\ \vdots \\ \mathbf{e}_{N(n-m)}^T \end{bmatrix}$$

is invertible and that for all $i \in N$, $x_i^* = 0$. Let

$$B = \{B(1), B(2), \dots, B(m)\} := \{1, 2, \dots, n\} - N$$

be an index set, then the above matrix can be re-arranged as

$$\begin{bmatrix} \mathbf{A}_B & \mathbf{A}_N \\ \mathbf{0} & \mathbf{I}_N \end{bmatrix},$$

which is invertible. Therefore, $\{\mathbf{A}_{B(1)}, \mathbf{A}_{B(2)}, \dots, \mathbf{A}_{B(m)}\}$ is linearly independent. $\square$

Note that according to Theorem 2.1.7, for every $i \in N$, $x_i = 0$. Note that $\mathbf{A} = \begin{bmatrix} \mathbf{A}_B & \mathbf{A}_N \end{bmatrix}$,

so the linear system $\mathbf{Ax} = \mathbf{b}$ is equivalent to $\mathbf{A}_B \mathbf{x}_B = \mathbf{b}$. Therefore, given any standard polyhedron $P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, we can construct a basic solution in P using the following procedures:

1. Choose m linearly independent columns from $\mathbf{A}$ with B being the index set for the columns to form the set $\{\mathbf{A}_{B(1)}, \mathbf{A}_{B(2)}, \dots, \mathbf{A}_{B(m)}\}$;
2. For each $i \in N := \{1, 2, \dots, n\} - B$, set $x_i = 0$. The vector consisting of all of these zero entries is denoted by $\mathbf{x}_N$;
3. Solve the linear system $\mathbf{A}_B \mathbf{x}_B = \mathbf{b}$ to obtain $\mathbf{x}_B = \mathbf{A}_B^{-1} \mathbf{b}$;
4. Let x_i be the i -th entry of $\mathbf{x}$, then

$$x_i = \begin{cases} 0, & \text{if } i \in N \\ (\mathbf{x}_B)_i & \text{if } i \in B \end{cases}.$$

The $\mathbf{x}$ obtained this way is alternatively denoted as $\mathbf{x} := (\mathbf{x}_B, \mathbf{x}_N)$.

Geometrically, 2 distinct basic feasible solutions of P are *adjacent* if there is an edge on the boundary joining the 2 points. Here we give an equivalent definition algebraically.

Definition 2.1.8 ► Adjacency

Let $\mathbf{x}_1$ and $\mathbf{x}_2$ be distinct basic solutions with respect to polyhedron P . $\mathbf{x}_1$ and $\mathbf{x}_2$ are said to be **adjacent** if there are exactly $(n - 1)$ linearly independent constraints active at both points, or their corresponding bases only contain 1 different basic column.

Recall that not all polyhedrons have basic feasible solutions. Informally, we can see that if a polyhedron does not contain any basic feasible solution, it contains at least 2 “openings” which allows us to place a straight line into the polyhedron. This observation is summarised rigorously as follows:

Theorem 2.1.9 ► Conditions for the Existence of Basic Feasible Solution

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{Ax} \leq \mathbf{b}\} \neq \emptyset,$$

then the following statements are equivalent:

1. P does not contain any straight line.
2. P has a basic feasible solution.
3. P has n linearly independent constraints.

Proof. Suppose P has a basic feasible solution $\mathbf{x}^*$, then there are n linearly independent constraints active at $\mathbf{x}^*$. Therefore, it is trivial that P must contain at least n linearly independent constraints.

We shall prove that (3) implies (1) by considering the contrapositive statement. Suppose that P contains a straight line $\{\mathbf{x}_0 + \lambda \mathbf{d} : \lambda \in \mathbb{R}\}$ for some fixed point $\mathbf{x}_0 \in P$ and direction vector $\mathbf{d} \in \mathbb{R}^n$, then for any $\lambda \in \mathbb{R}$, we have $\mathbf{A}(\mathbf{x}_0 + \lambda \mathbf{d}) \leq \mathbf{b}$. Notice that $\mathbf{A}\mathbf{x}_0 \leq \mathbf{b}$, so $\mathbf{A}\mathbf{d} = \mathbf{0}$. However, $\mathbf{d} \neq \mathbf{0}$, so $\mathbf{A}$ does not contain n independent rows, which implies that P does not have n linearly independent constraints.

Suppose that P does not contain any straight line. Take some $\mathbf{x} \in P$ and let

$$I(\mathbf{x}) := \{\mathbf{a}_i : \mathbf{a}_i^T \mathbf{x} = b_i\}$$

be the set of gradient vectors of all active constraints at $\mathbf{x}$. If $I(\mathbf{x})$ contains n linearly independent vectors, then we are done. Otherwise, $I(\mathbf{x})$ does not span $\mathbb{R}^n$, so there is some $\mathbf{d} \in \mathbb{R}^n$ with $\mathbf{d} \neq \mathbf{0}$ such that it is normal to $\text{span}(I(\mathbf{x}))$. Since P contains no straight lines, there exists some constraints with gradient vector $\mathbf{a}_j \notin I(\mathbf{x})$ such that we can find some $\mu \in \mathbb{R}$ such that $\mathbf{a}_j^T(\mathbf{x} + \mu \mathbf{d}) = b_j$. Set $\mathbf{x}' = \mathbf{x} + \mu \mathbf{d}$. Note that for all $\mathbf{a}_i \in I(\mathbf{x})$, we have $\mathbf{a}_i^T \mathbf{d} = 0$, and so $\mathbf{a}_i^T(\mathbf{x} + \mu \mathbf{d}) = b_i$. Therefore, $\mathbf{a}_i \in I(\mathbf{x}')$, and so we have $I(\mathbf{x}) \cup \{\mathbf{a}_j\} \subseteq I(\mathbf{x}')$. We claim that $\mathbf{a}_j$ linearly independent with $I(\mathbf{x})$. Otherwise, $\mathbf{a}_j^T \mathbf{d} = 0$, which means that the boundary defined by $\mathbf{a}_j^T \mathbf{x} = b_j$ is parallel to $\mathbf{d}$ and so if the constraint is active at $\mathbf{x}'$, it must also be active at $\mathbf{x}$, which is a contradiction. Therefore, $I(\mathbf{x}')$ has more linearly independent vectors than $I(\mathbf{x})$. Repeat this process and we will eventually obtain a set of n linearly independent gradient vectors, $I(\mathbf{x}^*)$, where $\mathbf{x}^*$ is a basic feasible solution. $\square$

Consider the level set defined by $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} = k$ for some affine function f . Suppose the level set has an intersection with some polyhedron P , then by translating the level set in the direction of $-\nabla f$, it will eventually intersect P at a basic feasible solution such that any further translation will make the intersection empty. Therefore, a reasonable guess is that an optimal solution for any linear program is closely linked to the basic feasible solutions of the feasible set.

We sketch the idea of the proof first. If P is a polyhedron representing the feasible set of a linear program, it is easy to check that the set of optimal solutions, $Q \subseteq P$, is also a polyhedron. We will attempt to pick a basic feasible solution $\mathbf{x}^*$ from Q first. Then, if $\mathbf{x}^*$ is not a basic feasible solution of P , we strategically choose some direction along which we can translate $\mathbf{x}^*$ to an interior point $\mathbf{y} \in P$ while still staying in Q . Once we are at $\mathbf{y}$, we

will be able to translate in the direction of $\mathbf{c}$ by a small distance to find a point where the function evaluates to a better value than the optimal value, arriving at a contradiction.

Theorem 2.1.10 ► Optimality of Basic Feasible Solutions

Consider the linear program

$$\min_{\mathbf{x} \in P} f(\mathbf{x})$$

where $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$ and P is a polyhedron. If P has a basic feasible solution and the linear program has an optimal solution, then there exists a basic feasible solution of P which is an optimal solution to the linear program.

Proof. Let the optimal value of f be v^* , then the set of optimal solutions is

$$Q := P \cap \{\mathbf{x} \in \mathbb{R}^n : \mathbf{c}^T \mathbf{x} = v^*\}.$$

Note that Q is a polyhedron. By Theorem 2.1.9, since P contains a basic feasible solution, it does not contain a straight line. Therefore, $Q \subseteq P$ cannot contain a straight line and so it contains a basic feasible solution.

Let $\mathbf{x}^*$ be any basic feasible solution of Q . We claim that $\mathbf{x}^*$ is a basic feasible solution of P . Suppose on contrary that $\mathbf{x}^*$ is not a basic feasible solution of P . Pick some $\mathbf{d} \in \mathbb{R}^n$ such that $\mathbf{c}^T \mathbf{d} = 0$. Let $P = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} \leq \mathbf{b}\}$. Notice that we can choose $\lambda \in \mathbb{R}$ such that $\mathbf{A}(\mathbf{x}^* + \lambda \mathbf{d}) < \mathbf{b}$, and so $\mathbf{x}^* + \lambda \mathbf{d} \in P$ is an internal point of P . However, this means that there exists some $\mu > 0$ such that

$$\mathbf{x}^* - \mu \mathbf{c} = \mathbf{x}^* + \lambda \mathbf{d} - (\mu \mathbf{c} + \lambda \mathbf{d}) \in P.$$

However, $\mathbf{c}^T(\mathbf{x}^* - \mu \mathbf{c}) < v^*$, which is impossible. Therefore, $\mathbf{x}^*$ must be a basic feasible solution of P . □

Note that this essentially implies that Q is the convex hull of basic feasible solutions of P . Therefore, any optimal solution of the original linear program can be expressed as a convex combination of basic feasible solutions of P , and so it suffices to first find all basic feasible solutions if our feasible set contains any.

2.2 The Simplex Method

Note that for any linear program, we can convert it to a standard linear program with feasible region

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}.$$

Let $\{\mathbf{x}_0 + \lambda \mathbf{d} : \lambda \in \mathbb{R}\}$ be any straight line, then clearly we can find some $\lambda \in \mathbb{R}$ such that there is some entry x_i of $\mathbf{x}$ with $x_i < 0$. Therefore, P does not contain any straight line, and so P always contains at least one basic feasible solution if $P \neq \emptyset$. Therefore, as long as a linear program has a finite optimal value, it has optimal solutions which are basic feasible solutions.

Therefore, to find the optimal solution of a linear program, we only need to start at any basic feasible solution and try to reach an adjacent basic feasible solution which can improve our objective value. Continue this search and eventually we will be able to collect all optimal solutions.

Given any basic feasible solution $\mathbf{x}$, we wish to find a direction $\mathbf{d}$ such that $\mathbf{x} + \theta \mathbf{d}$ is some adjacent basic feasible solution for some $\theta \in \mathbb{R}$. First, we need to ensure that $\mathbf{x} + \theta \mathbf{d}$ is still feasible.

Definition 2.2.1 ► Feasible Direction

Let P be a polyhedron and $\mathbf{x} \in P$ be a feasible point. A vector $\mathbf{d}$ is a **feasible direction** if $\mathbf{x} + \lambda \mathbf{d} \in P$ for some $\lambda > 0$.

Notice that $\mathbf{x} + \lambda \mathbf{d}$ is still feasible, so we must have $\mathbf{A}(\mathbf{x} + \lambda \mathbf{d}) = \mathbf{b}$. However, $\mathbf{A}\mathbf{x} = \mathbf{b}$, which implies that $\mathbf{A}\mathbf{d} = \mathbf{0}$. We can write $\mathbf{d} = (\mathbf{d}_B, \mathbf{d}_N)$ with respect to $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$. We need $\mathbf{x} + \lambda \mathbf{d} \geq \mathbf{0}$, but $\mathbf{x}_N = \mathbf{0}$, so we must also have $\mathbf{d}_N \geq \mathbf{0}$.

Suppose $\mathbf{d}$ is a feasible direction and $\mathbf{x}$ is a basic feasible solution. Consider $\mathbf{x}' = \mathbf{x} + \theta \mathbf{d} \in P$ for some $\theta \in \mathbb{R}$. If $\mathbf{x}'$ is on an edge, then clearly there is exactly one less active constraint at $\mathbf{x}'$ than at $\mathbf{x}$.

Theorem 2.2.2 ► Characterisation of A Direction Connecting Basic Feasible Solutions

Let $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$ with $\mathbf{x}_B \geq \mathbf{0}$ and $\mathbf{x}_N = \mathbf{0}$ be a basic feasible solution, then a direction that connects $\mathbf{x}$ to an adjacent basic feasible solution is in the form of $\mathbf{d}^j = (\mathbf{d}_B^j, \mathbf{d}_N^j)$ for some $j \in N$, such that $\mathbf{d}_N^j = \mathbf{e}_j$ and $\mathbf{d}_B^j = -\mathbf{A}_B^{-1} \mathbf{A}_j$.

Proof. Note that for any feasible $\mathbf{x}' = \mathbf{x} + \theta \mathbf{d}^j$ which is not a basic feasible solution, we have $\mathbf{A}\mathbf{x}' = \mathbf{b}$. This means that all constraints corresponding to B are active along the edge. Therefore, the edge frees one constraint of the form $x_j \geq 0$ for some $j \in N$. This means that the index set of active constraints along the edge is given by $B \cup N - \{j\}$. Note that

$$\mathbf{x} + \theta \mathbf{d}^j = (\mathbf{x}_B + \theta \mathbf{d}_B^j, \mathbf{x}_N + \theta \mathbf{d}_N^j).$$

Notice that $\mathbf{x}_N = \mathbf{0}$ and $\mathbf{x}_N + \theta \mathbf{d}_N^j = \theta \mathbf{e}_j$, so $\mathbf{d}_N^j = \mathbf{e}_j$. Since $\mathbf{x} + \theta \mathbf{d}^j$ is feasible, we

have

$$\mathbf{A}(\mathbf{x} + \theta \mathbf{d}^j) = \mathbf{b} = \mathbf{A}\mathbf{x}.$$

Therefore, $\mathbf{A}\mathbf{d}^j = \mathbf{0}$. However, note that

$$\begin{aligned}\mathbf{A}\mathbf{d}^j &= \begin{bmatrix} \mathbf{A}_B & \mathbf{A}_N \end{bmatrix} \begin{bmatrix} \mathbf{d}_B^j \\ \mathbf{d}_N^j \end{bmatrix} \\ &= \mathbf{A}_B \mathbf{d}_B^j + \mathbf{A}_N \mathbf{e}_j \\ &= \mathbf{A}_B \mathbf{d}_B^j + \mathbf{A}_j.\end{aligned}$$

Since $\mathbf{A}_B$ has linearly independent columns, it is invertible, so $\mathbf{d}_B^j = -\mathbf{A}_B^{-1} \mathbf{A}_j$. $\square$

Recall that the feasible polyhedron is actually the convex hull of all of its basic feasible solutions. Naturally, we may conjecture that any feasible direction is a linear combination of all the $\mathbf{d}^j$'s for $j \in N$.

Proposition 2.2.3 ▶ Feasible Directions as Linear Combinations

Let $\mathbf{x} := (\mathbf{x}_B, \mathbf{x}_N)$ be a basic feasible solution, then any feasible direction at $\mathbf{x}$ can be expressed as

$$\mathbf{d} = \sum_{j \in N} \lambda_j \mathbf{d}^j$$

for $\lambda_j \geq 0$.

Proof. Note that since $\mathbf{d}$ is a feasible direction, $\mathbf{A}\mathbf{d} = \mathbf{0}$. Since $\mathbf{A}\mathbf{d} = \mathbf{A}_B \mathbf{d}_B + \mathbf{A}_N \mathbf{d}_N$, this implies that

$$\mathbf{A}_B \mathbf{d}_B = -\mathbf{A}_N \mathbf{d}_N = -\sum_{j \in N} d_j \mathbf{A}_j.$$

Note that $\mathbf{A}_B$ is invertible, so by Theorem 2.2.2,

$$\mathbf{d}_B = -\sum_{j \in N} d_j \mathbf{A}_B^{-1} \mathbf{A}_j = \sum_{j \in N} d_j \mathbf{d}_B^j.$$

Since $\mathbf{d}_N^j = \mathbf{e}_j$, we have $\mathbf{d}_N = \sum_{j \in N} d_j \mathbf{d}_N^j$. Take $\lambda_j = d_j$, we have

$$\mathbf{d} = \sum_{j \in N} \lambda_j \mathbf{d}^j.$$

$\square$

Simply traversing between basic feasible solutions is not very useful, because our ultimate goal is to minimise our objective function, i.e., we wish to find a direction $\mathbf{d}^j$ such that for

some $\theta > 0$,

$$\mathbf{c}^T(\mathbf{x} + \theta \mathbf{d}^j) < \mathbf{c}^T \mathbf{x}.$$

Clearly, we require $\mathbf{c}^T \mathbf{d}^j < 0$. Note that

$$\begin{aligned} \mathbf{c}^T \mathbf{d}^j &= (\mathbf{c}_B, \mathbf{c}_N)^T \begin{bmatrix} \mathbf{d}_B^j \\ \mathbf{d}_N^j \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{c}_B^T & \mathbf{c}_N^T \end{bmatrix} \begin{bmatrix} -\mathbf{A}_B^{-1} \mathbf{A}_j \\ \mathbf{e}_j \end{bmatrix} \\ &= -\mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j + c_j, \end{aligned}$$

so our target is simply $c_j - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j < 0$.

Definition 2.2.4 ► Reduced Cost

Let $\mathbf{x}$ be a basic feasible solution with respect to objective function $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$. Let $\mathbf{c} = (\mathbf{c}_B, \mathbf{c}_N)$. For each $j = 1, 2, \dots, n$, the **reduced cost** of variable x_j is defined as

$$\bar{c}_j = c_j - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j.$$

Remark. For any $i \in B$, note that $\mathbf{A}_i = \mathbf{A}_B \mathbf{e}_i$, we have $\mathbf{A}_B^{-1} \mathbf{A}_i = \mathbf{e}_i$. Therefore, $\bar{c}_i = 0$.

We say that a direction $\mathbf{d}^j$ is an *improving direction* if and only if $\bar{c}_j < 0$. Furthermore, notice that by Theorem 2.2.2, we have

$$\begin{aligned} \bar{c}_j &= c_j - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j \\ &= \mathbf{c}^T \mathbf{e}_j - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j \\ &= \mathbf{c}_N^T \mathbf{d}_N^j - \mathbf{c}_B^T \mathbf{d}_B^j \\ &= \mathbf{c}^T \mathbf{d}^j. \end{aligned}$$

Combining with Proposition 2.2.3, the above gives us a quicker way to compute the reduced cost given any feasible direction at a point $\mathbf{x}$.

Next, we only need to determine a $\bar{\theta}_j > 0$ such that $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j$ gives us another basic feasible solution. Since we already know that $\mathbf{A}(\mathbf{x} + \bar{\theta}_j \mathbf{d}^j) = \mathbf{b}$ for all $\bar{\theta}_j > 0$, a natural idea is to take $\bar{\theta}_j$ to be the greatest positive real number such that $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j \geq \mathbf{0}$, i.e., we will move in the direction $\mathbf{d}^j$ until we can barely stay in the feasible polyhedron. Consider

$$\mathbf{x} + \bar{\theta}_j \mathbf{d}^j = (\mathbf{x}_B + \bar{\theta}_j \mathbf{d}_B^j, \mathbf{x}_N + \bar{\theta}_j \mathbf{d}_N^j).$$

Note that $\mathbf{x}_N \geq \mathbf{0}$ and $\bar{\theta}_j \mathbf{d}_N^j = \bar{\theta}_j \mathbf{e}_j \geq \mathbf{0}$, so it suffices to check that $\mathbf{x}_B + \bar{\theta}_j \mathbf{d}_B^j \geq \mathbf{0}$. This implies that for each $i \in B$, we have $\bar{\theta}_j \geq -\frac{x_i}{d_i^j}$. Therefore, we only need to take

$$\bar{\theta}_j = \min \left\{ -\frac{x_i}{d_i^j} : i \in B, d_i^j < 0 \right\}.$$

Note that here we do not consider those d_i^j 's with $d_i^j > 0$ because in such cases $x_i + \bar{\theta}_j d_i^j \geq 0$ for all $\bar{\theta}_j > 0$. Now, we would verify that the new point we reach is indeed another basic feasible solution.

Proposition 2.2.5 ► Criterion for Adjacent Basic Feasible Solutions

If $\{i \in B : d_i^j < 0\} \neq \emptyset$ and $\mathbf{x}$ is a basic feasible solution, then $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j$ is a basic feasible solution adjacent to $\mathbf{x}$.

Proof. Note that $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j$ is feasible, so $A(\mathbf{x} + \bar{\theta}_j \mathbf{d}^j) = \mathbf{b}$. Note that by the definition of $\bar{\theta}_j$, we have

$$\bar{\theta}_j = -\frac{x_\ell}{d_\ell^j}$$

for some $\ell \in B$. Therefore, we have $(\mathbf{x} + \bar{\theta}_j \mathbf{d}^j)_\ell = 0$. Notice that at $\mathbf{x} := (\mathbf{x}_B, \mathbf{x}_N)$, we have $\ell \in B$ and $j \in N$. Consider $\bar{B} := (B - \{\ell\}) \cup \{j\}$ and $\bar{N} := (N - \{j\}) \cup \{\ell\}$. One may check that $\bar{B}$ is linearly independent, so by Theorem 2.1.7, $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j$ is a basic feasible solution. $\square$

By now, we already have devised a systematic method to reach another basic feasible solution from a given basic feasible solution such that our objective value improves. The next goal is to determine a condition by which we should terminate our search and declare the current basic feasible solution to be optimal.

Before we do that, we need to first deal with an edge case where some basic feasible solution we find might be degenerate. Note that in the standard form, we have exactly m equality constraints represented by $A_B \mathbf{x}_B = \mathbf{b}$ which are always active, and at least $(n - m)$ active inequality constraints in the form of $\mathbf{x}_N = \mathbf{0}$. However, note that $\mathbf{x} \geq \mathbf{0}$ represents n inequality constraints, so in total we actually have $(m + n)$ constraints. An implication here is that if any entry of $\mathbf{x}_B$ is 0, then we will have more than n active constraints at $\mathbf{x}$.

Definition 2.2.6 ► Degeneracy in Standard Form

Let $\mathbf{x} = (\mathbf{x}_B, \mathbf{x}_N)$ be a basic feasible solution. We say that $\mathbf{x}$ is **degenerate** if there is some entry of $\mathbf{x}_B$ being 0.

Remark. This also concludes that $\mathbf{x}$ is non-degenerate if $\mathbf{x}_B = \mathbf{A}_B^{-1}\mathbf{b} > \mathbf{0}$.

Recall that along an improving direction $\mathbf{d}^j$, the reduced cost for x_j is $\bar{c}_j < 0$. A natural observation here is that if we cannot find any improving direction at a point $\mathbf{x}^*$, then this point must be optimal. Therefore, we are tempted to conclude that $\bar{c}_j \geq 0$ for all $j \in N$.

Theorem 2.2.7 ▶ Optimality Conditions for Simplex Method

Let $\mathbf{x}^* = (\mathbf{x}_B, \mathbf{x}_N)$ be a basic feasible solution to some standard linear program. Let $\bar{\mathbf{c}}$ be the vector of reduced costs associated with $\mathbf{x}^*$, then

1. If $\bar{\mathbf{c}} \geq \mathbf{0}$, then $\mathbf{x}^*$ is optimal;
2. If $\mathbf{x}^*$ is optimal and non-degenerate, then $\bar{\mathbf{c}} \geq \mathbf{0}$.

Proof. Suppose that $\bar{\mathbf{c}} \geq \mathbf{0}$. Let $\mathbf{y}$ be any feasible solution, then $\mathbf{y} - \mathbf{x}^*$ is a feasible direction. By Proposition 2.2.3,

$$\mathbf{y} - \mathbf{x}^* = \sum_{j \in N} \lambda_j \mathbf{d}^j,$$

where $\lambda_j \geq 0$ for all $j \in N$. Therefore,

$$\begin{aligned} \mathbf{c}^T \mathbf{y} &= \mathbf{c}^T \mathbf{x}^* + \sum_{j \in N} \lambda_j \mathbf{c}^T \mathbf{d}^j \\ &= \mathbf{c}^T \mathbf{x}^* + \sum_{j \in N} \lambda_j \bar{c}_j \\ &\geq \mathbf{c}^T \mathbf{x}^*. \end{aligned}$$

Therefore, $\mathbf{x}^*$ is an optimal solution.

Suppose that $\mathbf{x}^*$ is a non-degenerate optimal solution. For each $j \in N$, consider the adjacent basic feasible solution $\mathbf{x} + \bar{\theta}_j \mathbf{d}^j$. Notice that

$$\mathbf{c}^T \mathbf{x} \leq \mathbf{c}^T (\mathbf{x} + \bar{\theta}_j \mathbf{d}^j) = \mathbf{c}^T \mathbf{x} + \bar{\theta}_j \bar{c}_j$$

for all $j \in N$. Therefore, we must have $\bar{\theta}_j \bar{c}_j \geq 0$ for all $j \in N$. Since $\mathbf{x}$ is non-degenerate, for each $j \in N$ we have

$$\bar{\theta}_j = -\frac{x_\ell}{d_\ell^j} > 0,$$

and so $\bar{c}_j \geq 0$. Note that for all $i \in B$, we have $\bar{c}_i = 0$, so $\bar{\mathbf{c}} \geq \mathbf{0}$. □

Recall that $\bar{\mathbf{c}}_B = \mathbf{0}$, so it suffices to check that $\bar{\mathbf{c}}_N \geq \mathbf{0}$ for the $(n - m)$ inequality constraints to determine whether $\mathbf{x}^*$ is an optimal solution.

With the above preliminary works done, we are now able to develop the algorithm for simplex method.

Technique 2.2.8 ▶ Simplex Method

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ defined by $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$ be the objective function of a standard linear program with feasible set

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}.$$

The simplex method finds the optimal solution using the following procedures:

1. Initialise $\mathbf{x}_0$ to be any basic feasible solution.
2. At the k -th iteration, choose an index set B_k such that the columns of $\mathbf{A}_{B_k}$ are a basis for the column space of $\mathbf{A}$.
3. Let $N_k := \{1, 2, \dots, n\} - B_k$. For each $j \in N_k$, compute the reduced cost

$$\bar{c}_j = c_j - \mathbf{c}_{B_k}^T \mathbf{A}_{B_k}^{-1} \mathbf{A}_j.$$

4. If $\bar{c}_j \geq 0$ for all $j \in N_k$:
 - $\mathbf{x}_k$ is an optimal solution.
5. Otherwise:
 - (a) Take some $j \in N_k$ such that $\bar{c}_j < 0$. $(\mathbf{x}_k)_j$ is called an **entering variable**.
 - (b) Compute $\mathbf{d}_{B_k}^j = -\mathbf{A}_{B_k}^{-1} \mathbf{A}_j$.
 - (c) If $\mathbf{d}_{B_k}^j \geq \mathbf{0}$:
 - the problem is **unbounded**.
 - (d) Otherwise:
 - i. Let $\ell \in B_k$ be such that $\bar{\theta}_j = \min \left\{ -\frac{x_i}{d_i^j} : i \in B, d_i^j < 0 \right\} = \frac{x_\ell}{d_\ell^j}$. $(\mathbf{x}_k)_\ell$ is called a **leaving variable**.
 - ii. Update $B_{k+1} := (B - \{\ell\}) \cup \{j\}$.
 - iii. Update $\mathbf{x}_{k+1}$ by

$$(\mathbf{x}_{k+1})_i = \begin{cases} (\mathbf{x}_k)_i + \bar{\theta}_j \mathbf{d}_i^j & \text{if } i \in B - \{\ell\} \\ \bar{\theta}_j & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

2.3 Tableau Implementation

A common way to run simplex method is by tabulating the relevant variables, solutions and reduced costs so that we can keep track of their values conveniently. A generalised tableau looks like the following:

Basic	$\mathbf{x}$	Solution
$\bar{\mathbf{c}}$	$\mathbf{c}^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}$	$-\mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b}$
$\mathbf{x}_B$	$\mathbf{A}_B^{-1} \mathbf{A}$	$\mathbf{A}_B^{-1} \mathbf{b}$

If we expand the tableau with more detailed information, we get the following representation:

Basic	x_1	$\cdots$	x_n	Solution
$\bar{\mathbf{c}}$	$\bar{c}_1$	$\cdots$	$\bar{c}_n$	$-\mathbf{c}^T \mathbf{x}_B$
$x_{B(1)}$	$\mathbf{A}_B^{-1} \mathbf{A}_1 \quad \cdots \quad \mathbf{A}_B^{-1} \mathbf{A}_n$			$\mathbf{A}_B^{-1} \mathbf{b}$
$\vdots$				
$x_{B(i)}$				
$\vdots$				
$x_{B(m)}$				

Notice that $\mathbf{A}_B^{-1} \mathbf{A}_B = \mathbf{I}$, we may wish to re-arrange the columns of the tableau into the following form:

Basic	$\mathbf{x}_B$	$\mathbf{x}_N$	Solution
$\bar{\mathbf{c}}$	$\mathbf{0}$	$\mathbf{c}_N^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_N$	$-\mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b}$
$\mathbf{x}_B$	$\mathbf{I}$	$\mathbf{A}_B^{-1} \mathbf{A}_N$	$\mathbf{A}_B^{-1} \mathbf{b}$

Note that at each iteration, we need to “swap” the ℓ -th and j -th columns to update B . Essentially, suppose $\ell = B(i)$, then this is equivalent to performing row operations on the matrix

$$\begin{bmatrix} \mathbf{0} & \mathbf{c}_N^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_N & -\mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} \\ \mathbf{I} & \mathbf{A}_B^{-1} \mathbf{A}_N & \mathbf{A}_B^{-1} \mathbf{b} \end{bmatrix}$$

such that the j -th column becomes $\mathbf{e}_i$. Then, we will swap the row label $\mathbf{x}_\ell$ with the column label $\mathbf{x}_j$. We repeat this process until $\bar{\mathbf{c}} \geq \mathbf{0}$, where we will extract $\mathbf{A}_B^{-1} \mathbf{b}$ from the table as our $\mathbf{x}_B$.

2.4 Finding An Initial Basic Feasible Solution

Recall that to start the simplex method algorithm, we need to initialise $\mathbf{x}_0$ to be some basic feasible solution in the feasible region. However, this may not be straight-forward. Consider the standard linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} & f(\mathbf{x}) \\ \text{s.t. } & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

for some $\mathbf{A} \in \mathbb{R}^{m \times n}$.

We can choose the index set B such that $\mathbf{A}_B$ is invertible and so $\mathbf{A}^{-1}\mathbf{b}$ is a basic solution. However, we might not be so lucky that this basic solution is feasible, i.e., it is not easy to ensure that $\mathbf{A}^{-1}\mathbf{b} \geq \mathbf{0}$. To address this issue, we consider an *auxiliary linear program* as follows:

First, note that we can change $\mathbf{A}$ by multiplying some of its rows by -1 so that $\mathbf{b} \geq \mathbf{0}$. Then, we will introduce a new variable $\mathbf{y} \in \mathbb{R}^m$ and consider the linear program

$$\begin{aligned} \min_{\mathbf{y} \in \mathbb{R}^m} & \sum_{i=1}^m y_i \\ \text{s.t. } & \mathbf{Ax} + \mathbf{y} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{y} \geq \mathbf{0}. \end{aligned}$$

Now, it is obvious that $\mathbf{x} = \mathbf{0}$ and $\mathbf{y} = \mathbf{b}$ is a basic feasible solution to the problem. Solving this auxiliary minimisation problem will try to force $\mathbf{y} = \mathbf{0}$. If this is possible, i.e., the optimal value of the auxiliary program is 0, then it is clear that we have found some $\mathbf{x} \geq \mathbf{0}$ such that $\mathbf{Ax} = \mathbf{b}$, which is exactly a basic feasible solution to our original program!

Technique 2.4.1 ► Two-Phase Method

Consider a standard linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} & \mathbf{c}^T \mathbf{x} \\ \text{s.t. } & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

The two-phase method solves the linear program with the following procedures:

1. Multiplying some rows of $\mathbf{A}$ by -1 wherever necessary such that $\mathbf{b} \geq \mathbf{0}$.
2. Construct the auxiliary linear program

$$\begin{aligned} \min_{\mathbf{y} \in \mathbb{R}^m} \quad & \sum_{i=1}^m y_i \\ \text{s.t.} \quad & \mathbf{Ax} + \mathbf{y} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{y} \geq \mathbf{0}. \end{aligned}$$

3. Run simplex method on the auxiliary linear program to obtain its optimal solution $(\mathbf{y}^*, \mathbf{x}^*)$ and optimal value v^* .
4. If $v^* > 0$:
 - The original problem has empty feasible region.
5. Otherwise, $v^* = 0$:
 - Run simplex method on the original problem with $\mathbf{x}_0 = \mathbf{x}^*$.

Note that in the two-phase method, we need to solve 2 linear programs. This can be computationally expensive, so we wish to find a way to combine the two phases. Here we use the idea of a penalty term. Consider the linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^m} \quad & \mathbf{c}^T \mathbf{x} + M \sum_{i=1}^m y_i \\ \text{s.t.} \quad & \mathbf{Ax} + \mathbf{y} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{y} \geq \mathbf{0} \end{aligned}$$

where $M > 0$ is an arbitrarily large constant. We can see that if this augmented objective function has a finite optimal value, then it will also force $\mathbf{y} = \mathbf{0}$. The corresponding $\mathbf{x}$ will be the optimal solution of the original problem.

Technique 2.4.2 ► Big-M Method

Consider a standard linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

The big- M method solves the linear program with the following procedures:

1. Multiplying some rows of $\mathbf{A}$ by -1 wherever necessary such that $\mathbf{b} \geq \mathbf{0}$.
2. Augment the original problem with some arbitrarily large constant $M > 0$ into

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^m} \quad & \mathbf{c}^T \mathbf{x} + M \sum_{i=1}^m y_i \\ \text{s.t.} \quad & \mathbf{Ax} + \mathbf{y} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{y} \geq \mathbf{0}. \end{aligned}$$

3. Run simplex method on the augmented linear program.
4. If an optimal solution $(\mathbf{y}^*, \mathbf{x}^*)$ can be found with $\mathbf{y}^* = \mathbf{0}$:
 - $\mathbf{x}^*$ is an optimal solution to the original problem.
5. Otherwise:
 - The original problem has empty feasible set or is unbounded.

2.5 Special Cases

Next, we discuss a few special cases when using the simplex method. First, recall that in Definition 2.1.6, we know that a feasible basic solution $\mathbf{x}$ is degenerate if there is some redundant active constraint. However, note that this implies that more than one basic variables can leave. Algebraically, this means that there are different $\ell_1, \ell_2 \in B$ such that

$$\bar{\theta}_j = \min \left\{ -\frac{x_i}{d_i^j} : i \in B, d_i^j < 0 \right\} = -\frac{x_{\ell_1}}{d_{\ell_1}^j} = -\frac{x_{\ell_2}}{d_{\ell_2}^j}.$$

Without loss of generality, suppose we take x_{ℓ_1} as the leaving variable and try to update x_{ℓ_2} to x'_{ℓ_2} . By Proposition 2.2.5, we have

$$x'_{\ell_2} = x_{\ell_2} + \bar{\theta}_j d_{\ell_2}^j = x_{\ell_2} - \frac{x_{\ell_1}}{d_{\ell_1}^j} d_{\ell_2}^j = x_{\ell_2} - \frac{x_{\ell_2}}{d_{\ell_2}^j} d_{\ell_2}^j = 0,$$

so x_{ℓ_2} is indeed degenerate by Definition 2.2.6.

Note also that in some cases, we may have more than one optimal solution for the linear program. Intuitively, this occurs when at an optimal basic feasible solution, we can still find some feasible direction along which the objective value does not deteriorate. Therefore, **an alternative optimal solution can be found if and only if some reduced cost $\bar{c}_j = 0$ at an optimal solution.** There are 2 specific cases:

1. If we can find optimal solutions $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^k$, then the set of optimal solutions is bounded and can be constructed as

$$\text{conv}\{\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^k\}.$$

2. If at an optimal solution $\mathbf{x}^*$, there is $\bar{c}_j = 0$ but $\mathbf{d}^j \geq \mathbf{0}$, it means that $\mathbf{x}^* + \theta \mathbf{d}^j \geq \mathbf{0}$ is feasible for any $\theta \geq 0$, so the set of optimal solutions given by

$$\{\mathbf{x}^* + \theta \mathbf{d}^j : \theta \geq 0\}$$

is unbounded.

Similar to case 2 above, if at any basic feasible solution $\mathbf{x}^*$, there is some $j \in N$ such that $\bar{c}_j < 0$ and $\mathbf{d}^j \geq \mathbf{0}$, then any $\mathbf{x}^* + \theta \mathbf{d}^j$ where $\theta > 0$ is feasible and improves the objective value. This implies that the optimal value can be **improved indefinitely** and so the program is unbounded.

Lastly, suppose we have some linear program with an empty feasible set. We have seen in Section 2.4 that we can use either the two-phase method or the big- M method to detect this, because if the feasible set were to be empty, we will not be able to find an optimal solution for the auxiliary problem such that $\mathbf{y}^* = \mathbf{0}$.

Duality Theory

3.1 The Dual Problem

A *dual problem* can be seen as an alternative formulation of some linear program which is known as the *primal problem*. The motivation for the dual problem comes from the following observation: suppose we have a well-ordered set S and we wish to find $\inf S$. If it is difficult to minimise S directly, we may first consider a “relaxed” lower bound t , i.e., fix some t such that $t \leq s$ for all $s \in S$.

Then, by collecting all such lower bounds t into a set denoted as T , we know that for all $t \in T$, $t \leq s$ for all $s \in S$. Now, by **maximising** t , we obtain a **greatest possible lower bound** for S . If we can formulate the set T such that $S \cap T \neq \emptyset$, i.e., there is no x such that $\sup T < x < \inf S$, then this greatest lower bound is exactly $\inf S$. Consider the linear program

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \\ \text{s.t. } \mathbf{Ax} = \mathbf{b} \\ \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

The constraint $\mathbf{Ax} = \mathbf{b}$ can be re-written as $\mathbf{b} - \mathbf{Ax} = \mathbf{0}$. Now, we augment a “penalty” term $\mathbf{p}^T(\mathbf{b} - \mathbf{Ax})$ to the original objective function for some $\mathbf{p} \in \mathbb{R}^m$.

Proposition 3.1.1 ► Lower Bound for Optimal Value

Let $\mathbf{x}^*$ be an optimal solution to the linear program

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \\ \text{s.t. } \mathbf{Ax} = \mathbf{b} \\ \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

Define

$$g(\mathbf{p}) := \min_{\mathbf{x} \geq \mathbf{0}} \mathbf{c}^T \mathbf{x} + \mathbf{p}^T(\mathbf{b} - \mathbf{Ax}),$$

then $g(\mathbf{p}) \leq \mathbf{c}^T \mathbf{x}^*$.

Proof. Since $\mathbf{x}^* \geq \mathbf{0}$, we have

$$g(\mathbf{p}) = \min_{\mathbf{x} \geq \mathbf{0}} (\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbf{A}\mathbf{x})) \leq \mathbf{c}^T \mathbf{x}^* + \mathbf{p}^T (\mathbf{b} - \mathbf{A}\mathbf{x}^*).$$

Notice that $\mathbf{x}^*$ is feasible to the original problem, so $\mathbf{b} - \mathbf{A}\mathbf{x}^* = \mathbf{0}$. Therefore,

$$g(\mathbf{p}) \leq \mathbf{c}^T \mathbf{x}^* + \mathbf{p}^T (\mathbf{b} - \mathbf{A}\mathbf{x}^*) = \mathbf{c}^T \mathbf{x}^*.$$

□

Proposition 3.1.1 demonstrates a construction for a function g with respect to every linear program (P) such that g always bounds the objective value of (P) below.

Definition 3.1.2 ► Dual Problem

For any linear program

$$\begin{aligned} (P) \quad & \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \\ & \text{s.t. } \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

its **dual problem** is defined as

$$(D) \quad \max_{\mathbf{p} \in \mathbb{R}^m} \min_{\mathbf{x} \geq \mathbf{0}} (\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbf{A}\mathbf{x})),$$

and (P) is known as the **primal problem**.

Remark. We know that $g(\mathbf{p}) := \min_{\mathbf{x} \geq \mathbf{0}} (\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbf{A}\mathbf{x}))$ is an lower bound for the objective value of (P) , so (D) will try to search for the greatest lower bound of the optimal value of (P) .

Let us study this lower bound function $g(\mathbf{p})$ more closely. Observe that

$$g(\mathbf{p}) = \mathbf{p}^T \mathbf{b} + \min_{\mathbf{x} \geq \mathbf{0}} (\mathbf{c}^T - \mathbf{p}^T \mathbf{A}) \mathbf{x}.$$

Now we consider 2 cases. If $\mathbf{c}^T - \mathbf{p}^T \mathbf{A} \geq \mathbf{0}$, then it is clear that $\min_{\mathbf{x} \geq \mathbf{0}} (\mathbf{c}^T - \mathbf{p}^T \mathbf{A}) \mathbf{x} = 0$ by taking $\mathbf{x} = \mathbf{0}$. Otherwise, $(\mathbf{c}^T - \mathbf{p}^T \mathbf{A})_j < 0$ for some $j \in \mathbb{Z}^+$, then $(\mathbf{c}^T - \mathbf{p}^T \mathbf{A}) \mathbf{x}$ is unbounded because clearly for any feasible $\mathbf{x}$, we have $(\mathbf{c}^T - \mathbf{p}^T \mathbf{A})(\mathbf{x} + \mathbf{e}_j) < (\mathbf{c}^T - \mathbf{p}^T \mathbf{A}) \mathbf{x}$ and $\mathbf{x} + \mathbf{e}_j$ is obviously still feasible.

Therefore, we can essentially reduce $g(\mathbf{p})$ to

$$g(\mathbf{p}) = \begin{cases} \mathbf{p}^T \mathbf{b} & \text{if } \mathbf{c}^T - \mathbf{p}^T \mathbf{A} \geq \mathbf{0} \\ -\infty & \text{otherwise} \end{cases}.$$

Clearly, in order for us to be able to maximise $g(\mathbf{p})$, we need $\mathbf{c}^T - \mathbf{p}^T \mathbf{A} \geq \mathbf{0}$ to be satisfied by the dual problem. This means that we can have the following systematic construction for the dual problem:

Technique 3.1.3 ► Formulation of the Dual Problem

Intuitively, if we try to formulate the dual problem of the dual problem, then we should just return to the primal problem.

Proposition 3.1.4 ► Dual of the Dual Is the Primal

Let (P) be a primal problem with the dual problem (D) . If (D') is the dual problem of (D) , then (D') is equivalent to (P) .

3.2 Duality Theorems

We see that a valid dual problem is such that $\mathbf{p}^T \mathbf{A} \leq \mathbf{c}^T$.

Theorem 3.2.1 ► Weak Duality

Let

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

be a primal problem and consider its dual

$$\begin{aligned} \max_{\mathbf{p} \in \mathbb{R}^m} \quad & \mathbf{p}^T \mathbf{b} \\ \text{s.t.} \quad & \mathbf{p}^T \mathbf{A} \leq \mathbf{c}^T, \end{aligned}$$

then

$$\sup \mathbf{p}^T \mathbf{b} \leq \inf \mathbf{c}^T \mathbf{x}.$$

Proof. Let $\mathbf{p}$ and $\mathbf{x}$ be feasible, then $\mathbf{p}^T \mathbf{b} = \mathbf{p}^T \mathbf{A} \mathbf{x} \leq \mathbf{c}^T \mathbf{x}$, so $\sup \mathbf{p}^T \mathbf{b} \leq \inf \mathbf{c}^T \mathbf{x}$. $\square$

The above theorem justifies that the maximum of the dual objective is indeed an lower bound for the primal optimal value. We wish the primal and the dual problems to have an equal optimal value, so that solving one problem is equivalent to the other.

Corollary 3.2.2 ▶ Necessary Condition for Equal Optimal Values

Let $\mathbf{x}^$ and $\mathbf{p}^*$ be feasible solutions to a primal problem (P) and its dual problem (D). If $\mathbf{c}^T \mathbf{x}^* = (\mathbf{p}^*)^T \mathbf{b}$, then $\mathbf{x}^*$ and $\mathbf{p}^*$ are the optimal solutions.*

Proof. By Theorem 3.2.1,

$$(\mathbf{p}^*)^T \mathbf{b} \leq \sup \mathbf{p}^T \mathbf{b} \leq \inf \mathbf{c}^T \mathbf{x} \leq \mathbf{c}^T \mathbf{x}^*.$$

Since $\mathbf{c}^T \mathbf{x}^* = (\mathbf{p}^*)^T \mathbf{b}$, this implies that

$$(\mathbf{p}^*)^T \mathbf{b} = \sup \mathbf{p}^T \mathbf{b} = \inf \mathbf{c}^T \mathbf{x} = \mathbf{c}^T \mathbf{x}^*.$$

Therefore, $\mathbf{x}^*$ and $\mathbf{p}^*$ are the optimal solutions for the primal and dual problems respectively. $\square$

Another corollary helps us determine the boundedness and feasibility of a problem.

Corollary 3.2.3 ▶ Primal Is Unbounded If and Only If Dual Is Infeasible

If a primal problem (P) is unbounded, then its dual problem (D) is infeasible.

Proof. Suppose (P) is unbounded, then for every feasible $\mathbf{x}$, there exists some $\mathbf{x}'$ which is feasible such that $\mathbf{c}^T \mathbf{x}' < \mathbf{c}^T \mathbf{x}$. Suppose on contrary that there is a feasible solution $\mathbf{p}$ for (D), then by Theorem 3.2.1, $\mathbf{p}^T \mathbf{b} < \mathbf{c}^T \mathbf{x}$ for all feasible $\mathbf{x}$. However, this means that $\mathbf{c}^T \mathbf{x}$ has an infimum, which is a contradiction. $\square$

The opposite statement of the above corollary also holds, i.e., the primal is infeasible if its dual is unbounded.

Recall that in general, the duality gap between the primal and dual optimal values may not be 0, but in fact, all linear programs also gets for free *strong duality*.

Theorem 3.2.4 ▶ Strong Duality

If an linear program has an optimal solution, then so does its dual. Both the primal and the dual problems have the same optimal value.

Proof. Suppose that (P) is a primal linear program with an optimal solution $\mathbf{x}^*$, then there is some basis B such that $\mathbf{x}_B^* = \mathbf{A}_B^{-1}\mathbf{b}$. Since B is an optimal basis, we have

$$\bar{\mathbf{c}}^T = \mathbf{c}^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A} \geq \mathbf{0}.$$

Take $\mathbf{p}^T = \mathbf{c}_B^T \mathbf{A}_B^{-1}$, then $\mathbf{p}^T \mathbf{A} \leq \mathbf{c}^T$ and so $\mathbf{p}$ is feasible to the dual problem (D) . Note that $\mathbf{x}_N = \mathbf{0}$, so

$$\mathbf{p}^T \mathbf{b} = \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} = \mathbf{c}_B^T \mathbf{x}_B^* = \mathbf{c}^T \mathbf{x}^*.$$

By Corollary 3.2.2, $\mathbf{p}$ is an optimal solution for (D) , and so the dual optimal value is just $\mathbf{p}^T \mathbf{b} = \mathbf{c}^T \mathbf{x}^*$, which is the primal optimal value. $\square$

We would like to further characterise the primal and dual optimal solutions to a linear program. For that, we introduce the notion of *complementary slackness*.

Theorem 3.2.5 ▶ Complementary Slackness Conditions

Let (P) be a primal linear program with objective function $\mathbf{c}^T \mathbf{x}$, constraints $\mathbf{a}_i^T \mathbf{x} \leq b_i$, $\mathbf{a}_i^T \mathbf{x} \geq b_i$ or $\mathbf{a}_i^T \mathbf{x} = b_i$. Let $\mathbf{x}$ and $\mathbf{p}$ be feasible solutions to (P) and the dual problem (D) respectively, then $\mathbf{x}$ and $\mathbf{p}$ are optimal if and only if

$$p_i (\mathbf{a}_i^T \mathbf{x} - b_i) = 0, \quad (c_j - \mathbf{p}^T \mathbf{A}_j) x_j = 0$$

for all i, j .

Proof. Consider

$$\begin{aligned} \mathbf{c}^T \mathbf{x} - \mathbf{p}^T \mathbf{b} &= \mathbf{c}^T - \mathbf{p}^T \mathbf{A} \mathbf{x} + \mathbf{p}^T \mathbf{A} \mathbf{x} - \mathbf{p}^T \mathbf{b} \\ &= (\mathbf{c}^T - \mathbf{p}^T \mathbf{A}) \mathbf{x} + \mathbf{p}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) \\ &= \sum_j (c_j - \mathbf{p}^T \mathbf{A}_j) x_j + \sum_i p_i (\mathbf{a}_i^T \mathbf{x} - b_i). \end{aligned}$$

Note that $(c_j - \mathbf{p}^T \mathbf{A}_j) x_j, p_i (\mathbf{a}_i^T \mathbf{x} - b_i) \geq 0$ for all i, j . If $\mathbf{x}$ and $\mathbf{p}$ are optimal, by Theorem 3.2.4, $\mathbf{c}^T \mathbf{x} - \mathbf{p}^T \mathbf{b} = 0$ and so $(c_j - \mathbf{p}^T \mathbf{A}_j) x_j = p_i (\mathbf{a}_i^T \mathbf{x} - b_i) = 0$. If $(c_j - \mathbf{p}^T \mathbf{A}_j) x_j = p_i (\mathbf{a}_i^T \mathbf{x} - b_i) = 0$, then $\mathbf{c}^T \mathbf{x} - \mathbf{p}^T \mathbf{b} = 0$ and so by Theorem 3.2.1, $\mathbf{x}$ and $\mathbf{p}$ are optimal solutions. $\square$

Complementary slackness enables us to better characterise a primal optimal solution.

Proposition 3.2.6 ► Characterisation of Primal Optimal Solutions

Let $\mathbf{x}$ be a feasible solution to a primal linear program (P), then $\mathbf{x}$ is optimal if and only if there is a feasible dual solution $\mathbf{p}$ satisfying complementary slackness conditions.

Proof. Suppose that $\mathbf{x}$ is optimal, then by Theorem 3.2.4, there exists an optimal solution $\mathbf{p}^*$ to the dual problem (D). By Theorem 3.2.5, $\mathbf{p}^*$ satisfies complementary slackness. The converse is just Theorem 3.2.5. $\square$

3.3 Dual Simplex Method

In the previous section, we see that a dual problem helps us better characterise the optimal solutions for the primal problem. Consider a primal linear program (P) with its dual (D) in standard form. For any basis index set B , we have

$$\mathbf{x} = (\mathbf{A}_B^{-1}\mathbf{b}, \mathbf{0}), \quad \mathbf{p}^T = \mathbf{c}_B^T \mathbf{A}_B^{-1}.$$

Note that although $\mathbf{x}$ and $\mathbf{p}$ may not be both feasible, we still have

$$\mathbf{c}^T \mathbf{x} = \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} = \mathbf{p}^T \mathbf{b}.$$

By Corollary 3.2.2, $\mathbf{x}$ and $\mathbf{p}$ will be the optimal solutions if they are both feasible. This gives us inspirations to find the optimal solutions of a primal-dual pair, by either

1. maintain feasibility of $\mathbf{x}$ and work towards feasibility of $\mathbf{p}$, or
2. maintain feasibility of $\mathbf{p}$ and work towards feasibility of $\mathbf{x}$.

The two approaches are known as P-Algorithm and D-Algorithm respectively.

To maintain feasibility of $\mathbf{x}$, it is equivalent to maintaining $\mathbf{A}_B^{-1}\mathbf{b} \geq \mathbf{0}$. Now, for $\mathbf{p}$ to be feasible, we need $\mathbf{p}^T \mathbf{A} \leq \mathbf{c}^T$, which implies that

$$\bar{\mathbf{c}} = \mathbf{c}^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A} \geq \mathbf{0}.$$

This gives the implementation for the *primal algorithm*, which is exactly Technique 2.2.8.

Conversely, by maintaining $\bar{\mathbf{c}} \geq \mathbf{0}$, we only need to achieve $\mathbf{A}_B^{-1}\mathbf{b} \geq \mathbf{0}$ to make $\mathbf{x}$ feasible, because $\mathbf{A}\mathbf{x} = \mathbf{b}$ always holds. This leads to the *dual simplex method*.

Technique 3.3.1 ► Dual Simplex Method

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ defined by $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$ be the objective function of a standard linear program with feasible set

$$P := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}.$$

The dual simplex method finds the optimal solution using the following procedures:

1. Manipulate the constraints and add slack variables so that the right-most portion of $\mathbf{A}$ becomes an identity matrix.
2. Run simplex method on the transformed problem, while maintaining $\bar{\mathbf{c}} \geq \mathbf{0}$.
 - (a) At the k -th iteration, if there is no negative basic variable, then an optimal primal solution has been found.
 - (b) Otherwise, select some $x_\ell < 0$ as the leaving variable.
 - i. If $d_\ell^j \geq 0$ for all $j \in N_k$, then the primal problem is infeasible.
 - ii. Otherwise, take

$$i = \operatorname{argmin}_{j \in N_k} \left\{ \frac{\bar{c}_j}{|d_\ell^j|} : d_\ell^j < 0 \right\}$$

as the index of the entering variable.

3. Terminate the search when we have obtained $\mathbf{x}_B \geq \mathbf{0}$.

3.4 Sensitivity Analysis

Suppose that we have found an optimal solution $\mathbf{x}^*$ for some linear program, we are interested to know if the original problem is altered, how far away will $\mathbf{x}^*$ become from the new optimal solution. Specifically, we want to examine the sensitivity of

1. feasibility $\mathbf{A}_B^{-1} \mathbf{b} \geq \mathbf{0}$, and
2. optimality $\mathbf{c}^T - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A} \geq \mathbf{0}$.

Suppose $\mathbf{b}$ is changed to $\mathbf{b} + \delta \mathbf{e}_i$. Note that the optimality condition is independent of $\mathbf{b}$ and so is unaffected. If $\mathbf{x}^*$ were still to be feasible, we must have

$$\mathbf{0} \leq \mathbf{A}_B^{-1}(\mathbf{b} + \delta \mathbf{e}_i) = \mathbf{x}_B^* + \delta (\mathbf{A}_B^{-1} \mathbf{e}_i).$$

The above inequality yields a range of values for δ such that $\mathbf{x}^*$ remains optimal and feasible in the altered problem. The optimal value in the altered problem is given by

$$\begin{aligned}\mathbf{c}_B^T \mathbf{A}_B^{-1} (\mathbf{b} + \delta \mathbf{e}_i) &= \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} + \delta \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{e}_i \\ &= \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} + \delta (\mathbf{p}^*)^T \mathbf{e}_i \\ &= \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{b} + \delta p_i^*.\end{aligned}$$

We see that the change in objective value is closely tied to the dual optimal solution, such that a small change of δ in b_i results in a change of δp_i^* in the optimal value.

Definition 3.4.1 ► Marginal Cost

Let (P) be a primal problem with dual problem (D) . If the dual problem has an optimal solution $\mathbf{p}^*$, then p^* is called the **marginal cost** or **shadow cost** of b_i for minimisation (P) , and the **marginal profit** or **shadow price** of b_i for maximisation (P) .

Suppose that $\mathbf{c}$ is changed to $\mathbf{c} + \delta \mathbf{e}_j$ for some j . Note that the feasibility condition is independent of $\mathbf{c}$. If x_j is non-basic, the altered reduced cost is

$$\bar{c}'_j = c_j + \delta - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_j = \bar{c}_j + \delta.$$

Clearly, optimality remains if and only if $\delta \geq -\bar{c}_j$. If x_j is basic, then for each $i \in N$, we need

$$\begin{aligned}c_i - (\mathbf{c}_B + \delta \mathbf{e}_j)^T \mathbf{A}_B^{-1} \mathbf{A}_i &= c_i - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_i - \delta \mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i \\ &= \bar{c}_j - \delta \mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i \\ &\geq 0.\end{aligned}$$

If $\mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i > 0$, we have $\delta \leq \frac{\bar{c}_j}{\mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i}$. If $\mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i < 0$, we have $\delta \geq \frac{\bar{c}_j}{\mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i}$. If $\mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i = 0$, it is clear that $\bar{c}_j$ is unchanged. Therefore, let $\bar{a}_{i,j} = \mathbf{e}_j^T \mathbf{A}_B^{-1} \mathbf{A}_i$, we have

$$\max_{\bar{a}_{i,j} < 0} \frac{\bar{c}_j}{\bar{a}_{i,j}} \leq \delta \leq \min_{\bar{a}_{i,j} > 0} \frac{\bar{c}_j}{\bar{a}_{i,j}}.$$

Suppose that some entry a_{ij} of a non-basic column $\mathbf{A}_j$ is changed to $a_{ij} + \delta$ for some $\delta \neq 0$. Note that $j \notin B$, so this change does not affect the feasibility $\mathbf{A}_B^{-1} \mathbf{b} \geq \mathbf{0}$. In terms of

optimality, the only reduced cost affected is

$$\begin{aligned}\bar{c}'_j &= c_j - \mathbf{c}_B^T \mathbf{A}_B^{-1} (\mathbf{A}_j + \delta \mathbf{e}_i) \\ &= \bar{c}_j - \delta \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{e}_i \\ &= \bar{c}_j - \delta p_i.\end{aligned}$$

Clearly, if $\bar{c}_j - \delta p_i \geq 0$, the original optimal solution remains optimal in the altered problem, so we obtain a condition on δ to maintain the optimality.

Suppose we add a new variable x_{n+1} , i.e., we raise the dimension of the problem by 1, then we obtain a new problem

$$\begin{aligned}\min \quad & \mathbf{c}^T + c_{n+1}x_{n+1} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} + x_{n+1}\mathbf{A}_{n+1} = \mathbf{b}, \\ & \mathbf{x} \geq \mathbf{0}, \quad x_{n+1} \geq 0.\end{aligned}$$

Note that $(\mathbf{x}^*, 0)$ is still a basic feasible solution to the new problem by taking $x_{n+1} = 0$, so it suffices to consider the optimality at $(\mathbf{x}^*, 0)$ by computing

$$\bar{c}_{n+1} = c_{n+1} - \mathbf{c}_B^T \mathbf{A}_B^{-1} \mathbf{A}_{n+1}.$$

If $\bar{c}_{n+1} \geq 0$, this new basic feasible solution is still an optimal solution. Otherwise, this implies that x_{n+1} can be an entering variable, and we should use the simplex method to find the optimal solution for the update problem.

Suppose we add a new constraint $\mathbf{a}_{m+1}^T \mathbf{x} \leq b_{m+1}$, we will actually implicitly introduce a new variable x_{n+1} as a slack variable. Note that the objective function remains unchanged, so if $\mathbf{x}^*$ satisfies this new constraint, there is nothing to do — it is still an optimal solution!

Otherwise, $\mathbf{x}^*$ becomes infeasible. Note that now we should have one more basic solution due to the new constraint. To find the new optimal, we first consider the new constraint in standard form:

$$\mathbf{a}_{m+1}^T \mathbf{x} + x_{n+1} = b_{m+1}.$$

Suppose we add x_{n+1} as a new basic variable. Since $\mathbf{x}^*$ is infeasible now in the altered problem, we have $\mathbf{a}_{m+1}^T \mathbf{x} > b_{m+1}$, so $x_{n+1} < 0$ is infeasible.

Network Optimisation

4.1 Network Flow Problems

In network flow problems, we study directed movement of resources between nodes. It is easy to see that such problems can be easily modelled with *directed graphs*.

Definition 4.1.1 ► Directed Graph

A **directed graph** $G = (V, E)$ consists of a vertex set $V(G)$ and an edge set $E(G)$, where $E(G)$ consists of ordered pairs of vertices in $V(G)$.

Every directed graph G induces an underlying undirected graph G' by defining

$$V(G') := V(G), \quad E(G') := \{uv : \{(u, v), (v, u)\} \cap E(G) \neq \emptyset\}.$$

We say that G is *connected* if and only if G' is connected.

Definition 4.1.2 ► Network

A **network** is a directed graph G with a mapping $x : E(G) \rightarrow \mathbb{R}_0^+$ known as the **flow**, a mapping $c : E(G) \rightarrow \mathbb{R}_0^+$, known as the **cost**, and a mapping $b : V(G) \rightarrow \mathbb{R}$, known as the **external supply/demand**.

If $x_{ij} = x(u_i u_j) \leq u_{ij}$ for some *upper bound* u_{ij} , we say that the network is *capacitated*. Otherwise, we say that the corresponding problem is *uncapacitated*. For any $v \in V(G)$, we say that v is a

- *supply node* if $b(v) > 0$;
- *demand node* if $b(v) < 0$;
- *trans-shipment node* if $b(v) = 0$.

We also denote $b(v) = b_v$.

Note that the above construction can be easily applied to directed multigraphs by merging all edges (u, v) and consider the aggregate flow between the vertices.

In network optimisation, we wish to maintain *flow balance*, i.e., whatever flows out of a node (total flow out plus external demand) must equal the sum of total flow in and external

supply.

Definition 4.1.3 ► Flow Balance Constraint

Let G be a network. For each $v \in V(G)$, denote

$$O(v) := \{u \in V(G) : (v, u) \in E(G)\}, \quad I(v) := \{u \in V(G) : (u, v) \in E(G)\},$$

then the **flow balance constraint** is formulated as

$$\sum_{u \in O(v)} x_{vu} - \sum_{u \in I(v)} x_{uv} = b_v$$

for all $v \in V(G)$.

We say that a network has a feasible flow if it satisfies the flow balance constraint and the *capacity constraint*. Intuitively, this means that

$$\sum_{v \in V(G)} b_v = 0.$$

We can prove this equality more rigorously by considering the following representation:

Definition 4.1.4 ► Node-Arc Incidence Matrix

Let G be a simple directed graph. Label the vertices

$$V(G) = \{v_1, v_2, \dots, v_n\}$$

and edges

$$E(G) = \{e_j = (u_j, w_j) : j = 1, 2, \dots, m\}.$$

The matrix $\mathbf{A}$ defined by

$$a_{ij} = \begin{cases} 1 & \text{if } v_i = u_j \\ -1 & \text{if } v_i = w_j \\ 0 & \text{otherwise} \end{cases}$$

is said to be the **node-arc incidence matrix** of G .

Clearly, the flow for every $e_j \in E(G)$ is just $x_{u_j w_j}$. Collect the flows on all edges into a $1 \times m$ column vector

$$\mathbf{x} = \begin{bmatrix} x_{u_1 w_1} & x_{u_2 w_2} & \cdots & x_{u_m w_m} \end{bmatrix}^T,$$

then $b_{v_i} = (\mathbf{A}^T)_i \mathbf{x}$, where $(\mathbf{A}^T)_i$ is the i -th row of $\mathbf{A}$. Observe that the column sum of $\mathbf{A}$ is $\mathbf{0}$,

which naturally implies that

$$\sum_{v \in V(G)} b_v = \left(\sum_{i=1}^n (\mathbf{A}^T)_i \right) \mathbf{x} = 0.$$

Notice that by using the node-arc incidence matrix, we can actually formulate a network flow problem as the linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \\ \text{s.t. } \mathbf{Ax} = \mathbf{b} \\ \mathbf{0} \leq \mathbf{x} \leq \mathbf{u}, \end{aligned}$$

where $\mathbf{x}$ is the flow vector, $\mathbf{c}$ is the cost, $\mathbf{b}$ is the external supply vector, $\mathbf{A}$ is the node-arc incidence matrix and $\mathbf{u}$ is the upper bound for flow.

In general, there are some types of linear programs which can be converted to a network flow problem. For example, if the constraint of a linear program is of the form $\mathbf{Ax} = \mathbf{b}$ where $\mathbf{A}$ is an $m \times n$ 0-1 matrix such that the 1's in each column appear consecutively, then we can obtain a corresponding node-arc incidence matrix by

$$\mathbf{A}' = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 0 \\ -1 & 1 & 0 & \cdots & 0 & 0 \\ 0 & -1 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & -1 & 1 \\ 0 & 0 & 0 & \cdots & 0 & -1 \end{bmatrix} \mathbf{A}.$$

4.1.1 Single-Source Shortest Path Problem

In graph theory, the *single-source shortest path problem* (SSSP) is a classic and easy-to-solve optimisation problem. The problem statement is as follows:

Let G be a directed graph and $w : E(G) \rightarrow \mathbb{R}$ be a weight mapping on edges. Fix a source vertex s . For any destination vertex $t \in V(G)$, find an (or all) s - t path(s) P_{st} in G such that

$$\sum_{v \in V(P_{st})} w(v)$$

is minimised.

In programming, an SSSP can be easily solved by deploying one of many greedy graph traversal algorithms, such as Bellman-Ford Algorithm, Dijkstra's Algorithm and A* Algorithm, provided that the graph does not contain any negatively weighted cycle, where the problem is unsolvable.

How to formulate an SSSP as a linear program? Suppose s is our source and t is our destination, the shortest s - t path is essentially such that **the sum of flow costs over the edges in the path is minimised**. Therefore, it suffices to send a **unit flow from s to t** and minimise the total cost $\mathbf{c}^T \mathbf{x}$.

Proposition 4.1.5 ▶ SSSP as a Linear Program

Let G be a network with source and destination vertices s and t respectively. Let $\mathbf{x}^*$ be an optimal solution to the linear program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad \sum_{u \in O(v)} x_{vu} - \sum_{u \in I(v)} x_{uv} = \begin{cases} 1 & \text{if } v = s \\ -1 & \text{if } v = t \\ 0 & \text{otherwise} \end{cases} \\ \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

such that $x_i^* \in \{0, 1\}$, then the shortest s - t path is induced by $\{e_i \in E(G) : x_i^* = 1\}$.

Proof. The case where $s = t$ is trivial. Assume $s \neq t$. Let P be the subgraph of G induced by $E(P) := \{e_i \in E(G) : x_i^* = 1\}$. We first prove that P is an s - t walk in G . It is easy to see that $t \in V(P)$, because otherwise

$$\sum_{u \in O(t)} x_{tu} - \sum_{u \in I(t)} x_{ut} = 0.$$

Similarly, $s \in V(P)$. We claim that s and t are the only end vertices in P . Suppose on contrary that there exists some $r \in V(P)$ with $r \neq s, t$ and either $O(r) = \emptyset$ or $I(r) = \emptyset$. However,

$$\sum_{u \in O(r)} x_{ru} - \sum_{u \in I(r)} x_{ur} = 0 \neq -1,$$

so r must be an isolated vertex in P , which is a contradiction because P is induced on a subset of $E(G)$, meaning that every connected component of P must contain an edge. Therefore, P is an s - t walk. Suppose on contrary that P is not a shortest s - t path,

then there exists a shorter s - t path P' in G . Consider $\mathbf{x}^{**}$ defined by

$$x_i^{**} = \begin{cases} 1 & \text{if } e_i \in E(P') \\ 0 & \text{otherwise} \end{cases},$$

then $\mathbf{c}^T \mathbf{x}^{**} < \mathbf{c}^T \mathbf{x}^*$, which contradicts the optimality of $\mathbf{x}^*$. Therefore, P is a shortest s - t path in G . □

An alternative perspective to view Proposition 4.1.5 is as follows: label the edge set

$$E(G) := \{e_1, e_2, \dots, e_m\}.$$

Define a mapping $E(G) \mapsto \mathcal{B}$, where $\mathcal{B}$ is the family of all boolean arrays of length m , as follows: for every edge $e_i \in E(G)$, mark it as “taken” by mapping it to 1 and “not taken” by mapping it to 0. The corresponding boolean array $\mathbf{x}$ optimal to the minimisation problem essentially indicates all edges included in the shortest s - t path in G .

In addition, we can re-write the SSSP problem in the following form:

$$\begin{aligned} \min \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} = \mathbf{e}_s - \mathbf{e}_t \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

the dual problem to an SSSP can be formulated as

$$\begin{aligned} \max \quad & \mathbf{p}^T (\mathbf{e}_s - \mathbf{e}_t) \\ \text{s.t.} \quad & \mathbf{A}^T \mathbf{p} \leq \mathbf{c}, \end{aligned}$$

which can be further simplified to

$$\begin{aligned} \max \quad & p_s - p_t \\ \text{s.t.} \quad & p_i - p_j \leq c_{(i,j)} \quad \text{for all } (i, j) \in E(G). \end{aligned}$$

This means that $\mathbf{p}$ and $\mathbf{p} + \boldsymbol{\alpha}$ are equivalent in the dual problem and so we can safely set $p_t = 0$ by defining $\boldsymbol{\alpha} = -p_t$.

We now discuss a few applications of SSSP programs.

Three Jug Puzzle

Three Jug Puzzle

Given a a -litre jug filled with water and two empty jugs of capacity b and c litres respectively such that $a = b + c$, find the minimum number of steps to measure out exactly d litres of water by transferring water between the jugs only.

We first define a state in the problem by an ordered 3-tuple (a_i, b_i, c_i) indicating the amount of water in the three jugs respectively. Clearly, $a_i + b_i + c_i = a$ and so a_i is actually implicitly determined by b_i and c_i . Therefore, every state in the problem can be represented by an ordered pair (b_i, c_i) . Now, construct a network with vertex set

$$V := \{(i, j) : i = 0, 1, \dots, b, j = 0, 1, \dots, c\}.$$

We define the edges as follows:

- $(i, j) \rightarrow (b, j)$: transferring water from the first jug to the second jug;
- $(i, j) \rightarrow (0, j)$: transferring water from the second jug to the first jug;
- $(i, j) \rightarrow (i, c)$: transferring water from the first jug to the third jug;
- $(i, j) \rightarrow (i, 0)$: transferring water from the third jug to the first jug;
- $(i, j) \rightarrow (\min\{0, i + j - c\}, \max\{i + j, c\})$: transferring water from the second jug to the third jug;
- $(i, j) \rightarrow (\max\{i + j, b\}, \min\{0, i + j - b\})$: transferring water from the third jug to the second jug;

Now we only need to find the shortest $(0, 0)$ - (i, j) path such that either $i = d$ or $j = d$ or $a - i - j = d$.

Dynamic Lot Sizing

Dynamic Lot Sizing

Suppose a factory has T production periods with demand d_i for the i -th period. Let x_i be the output of the i -th period and I_i be the inventory at the end of the $(i - 1)$ -th period. The cost per unit of production is c_i and the cost per unit of inventory holding is h_i in the i -th period, and a set-up cost of K_i is incurred in the i -th period whenever $x_i > 0$. Determine a production plan $\mathbf{x} = (x_1, x_2, \dots, x_T)$ to meet the demands and minimise the total cost.

First, notice that $I_i = I_{i-1} + x_i - d_i$. To minimise the total cost, we should not produce any redundant unit of goods, and so $I_0 = I_T = 0$. Moreover, the following proposition is true:

Proposition 4.1.6 ▶ Optimality Condition of Dynamic Lot Sizing

If $(x_1, x_2, \dots, x_T)$ is an optimal production plan, then $I_{i-1}x_i = 0$ and $x_i = \sum_{k=i}^{j_i} d_k$ for some $j_i \geq i - 1$.

This means that to achieve minimal production cost, we should not both use the inventory and produce new goods in a period, and each production should be prudently scheduled to meet future demands for several periods. Therefore, suppose there are m periods, labelled as $j_1, j_2, \dots, j_m$, where production is carried out, then

$$x_{j_k} = \sum_{i=j_k}^{j_{k+1}-1} d_i.$$

Therefore, if we define the T periods as a vertex set, then a 0 - T path indicates the periods where production should take place, where an edge (i, j) connects two consecutive production periods. Therefore, we can define the flow cost on the edge by

$$c_{(i,j)} = K_i + c_i x_i + \sum_{k=i}^{j-1} h_k I_k,$$

where

$$I_k = x_i - \sum_{r=i}^k d_r = \sum_{r=k+1}^{j-1} d_r.$$

Since we do not carry over I_{j-1} to the j -th period, we need $I_{j-1} = 0$ to minimise wastage. Therefore,

$$c_{(i,j)} = K_i + c_i \sum_{k=i}^{j-1} d_k + \sum_{k=i}^{j-2} \left(h_k \sum_{r=k+1}^{j-1} d_r \right).$$

Now, we only need to find the shortest 0 - T path.

Project Management

Project Management

Given a list of activities $A_1, A_2, \dots, A_n$. Let t_i be the duration for A_i and $P(A_i)$ be the required activities which must be completed before A_i starts. For each activity, find the maximum possible time to delay without affecting the project completion time.

For each activity A_i , write $A_i := (s_{E_i}, e_{E_i}, s_i, l, s_{L_i}, e_{L_i})$ as a node, where s_{E_i}, e_{E_i} are the earli-

est start and end time and s_{L_i}, e_{L_i} are the latest start and end time, such that $s_i = s_{L_i} - s_{E_i}$. s_i is known as the *slack* of A_i . Connect $A_i \rightarrow A_j$ if $A_i \in P(A_j)$. Take a node with the lowest topological ordering as the source node. We will first process the nodes in the forward direction and set

$$s_{E_i} = \max \{e_{E_j} : A_j \in P(A_i)\},$$

i.e., take the latest start time for each activity. After all nodes are processed, we backtrack from the node with the highest topological ordering and set

$$e_{L_i} = \min \{s_{L_j} : A_i \in P(A_j)\}.$$

Eventually, there will be a path such that every node in the path has zero slack. This path is known as the *critical path*.

4.1.2 Maximum Flow Problem

The maximum flow problem has the following statement:

Let G be a capacitated network with u_{ij} as the upper bound for the flow on edge (i, j) . For a fixed source vertex s and a fixed destination vertex t , find the maximum external supply b_s at s such that we can transport b_s units of flow from s to t .

Suppose $b_s = b$, then the maximum flow problem can be formulated as the linear program

$$\begin{aligned} \min & b \\ & b \geq 0 \\ \text{s.t.} \quad & \sum_{u \in O(v)} x_{vu} - \sum_{u \in I(v)} x_{uv} = \begin{cases} b & \text{if } v = s \\ -b & \text{if } v = t \\ 0 & \text{otherwise} \end{cases} \\ & \mathbf{0} \leq \mathbf{x} \leq \mathbf{u}. \end{aligned}$$

The flow balance constraint can be written as $\mathbf{Ax} = \mathbf{df}$, where f is the amount of flow from s to t and $\mathbf{d} = \mathbf{e}_s - \mathbf{e}_t$.

A variant of the maximum flow problem is the multi-source maximum flow problem, where we are given n source nodes $s_1, s_2, \dots, s_n$ and m destination nodes $t_1, t_2, \dots, t_m$. The aim is to find the maximum total flow from the sources to the destinations. This can be easily converted back to the single-source case by adding a new source s and a new destination t , and join $s \rightarrow s_i$ for all $i = 1, 2, \dots, n$ and $t_j \rightarrow t$ for all $j = 1, 2, \dots, m$ using uncapacitated arcs.

A problem closely related to the maximum flow problem is the *minimum cut problem*, which states the following:

Let G be a directed graph with $w : E(G) \rightarrow \mathbb{R}_0^+$ as a weight map over the edges. For two fixed vertices $s, t \in V(G)$, find an edge cut $X \subseteq E(G)$ with minimum total weight such that s and t are in different connected components in $G - X$.

We introduce a few relevant definitions to the minimum cut problem.

Definition 4.1.7 ▶ s - t Cut

Let G be a graph and $s, t \in V(G)$. An **s - t cut** in G is a bipartition $\{S_1, S_2\}$ of $V(G)$ such that $s \in S_1$ and $t \in S_2$. The cut is minimal if $|E(S_1, S_2)|$ is minimised.

Remark. Clearly, the total capacity of the minimal s - t cut in G upper bounds the maximum flow from s to t in G .

Therefore, this motivates us to view a maximum flow problem from the perspective of a minimum cut problem. To formulate a minimum s - t cut problem, we can assign boolean flags to the vertices, such that $y_i = 1$ if $u_i \in S_1$ and $y_i = 0$ if $u_i \in S_2$. It is clear that $y_s = 1$ and $y_t = 0$.

For every edge $u_i u_j$, we again assign boolean flags to them such that $z_{ij} = 1$ if $u_i u_j$ is an S_1 - S_2 edge and $z_{ij} = 0$ otherwise. Additionally, note that for every edge $u_i u_j$, if $z_{ij} = 1$, we can without loss of generality assume that $y_i = 1$ and $y_j = 0$, and so $y_i - y_j = 1$. Otherwise, $z_{ij} = 0$ and $y_i = y_j$. Either way, we have $y_i - y_j \leq z_{ij}$. However, if $\mathbf{A}$ is the node-arc incident matrix, $y_i - y_j$ is exactly the row corresponding to $u_i u_j$ in $\mathbf{A}^T \mathbf{y}$.

Therefore, we can formulate the minimum s - t cut problem as follows:

$$\begin{aligned} \min \quad & \mathbf{u}^T \mathbf{z} \\ \text{s.t.} \quad & \mathbf{d}^T \mathbf{y} = 1 \\ & -\mathbf{A}^T \mathbf{y} + \mathbf{z} \leq \mathbf{0} \\ & \mathbf{z} \geq \mathbf{0} \end{aligned}$$

With some careful work, it can be verified that this is exactly the dual problem of a maximum flow problem. By Theorem 3.2.4, this means that the total capacity of the minimum s - t cut gives the maximum s - t flow.

Proposition 4.1.8 ▶ Maximum Flow Problem and Minimum Cut Problem

The maximum s - t flow in a network G is equal to the total capacity of the minimum s - t cut in G .

4.2 Network Simplex Method

Consider an uncapacitated minimum cost flow problem formulated as

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^m} \mathbf{c}^T \mathbf{x} \\ \text{s.t. } \mathbf{A} \mathbf{x} = \mathbf{b} \\ \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

Note that here, $\mathbf{A}$ is a node-arc incidence matrix. By Definition 4.1.4, we see that $\mathbf{A}$ has a very nice structure, which makes such network problems to be solvable very efficiently. We will modify the simplex method to solve such problems.

We first characterise basic feasible solutions in the problem. Take any index set B . Notice that the row sum of $\mathbf{A}_B$ is always 0, so any $\mathbf{A}_B$ is never invertible. Therefore, we can never find a basis for the network problem and thus cannot compute the basic solution using $\mathbf{x}_B = \mathbf{A}_B^{-1} \mathbf{b}$.

Definition 4.2.1 ▶ Truncated Node-Arc Incidence Matrix

Let $\mathbf{A}$ be a node-arc incidence matrix, the **truncated node-arc incidence matrix** $\tilde{\mathbf{A}}$ is obtained by deleting any row from $\mathbf{A}$.

It is easy to check that the rows of $\tilde{\mathbf{A}}$ are always linearly independent, provided that the underlying directed graph has no isolated vertex. Correspondingly, we can obtain a truncated constraint $\tilde{\mathbf{A}} \mathbf{x} = \tilde{\mathbf{b}}$. Now, for any network G , we consider its spanning trees.

Definition 4.2.2 ▶ Tree Solution

Let G be a network with node-arc incidence matrix $\mathbf{A}$. A flow vector $\mathbf{x} \in \mathbb{R}^{e(G)}$ is a **tree solution** if $\tilde{\mathbf{A}} \mathbf{x} = \tilde{\mathbf{b}}$ and there exists some spanning tree T of G such that $x_i = 0$ for all $e_i \notin E(T)$. A **feasible tree solution** is a tree solution $\mathbf{x}$ with $\mathbf{x} \geq \mathbf{0}$.

Proposition 4.2.3 ▶ Spanning Tree Induces Basis on $\tilde{\mathbf{A}}$

Let $\mathbf{A}$ be the node-arc incidence matrix for a network G . An index set B is a basis of $\tilde{\mathbf{A}}$ if and only if the set $E_B := \{e_i \in E(G) : i \in B\}$ induces a spanning tree of G .

Proof. Without loss of generality, suppose $\tilde{\mathbf{A}}$ is the first $(n - 1)$ rows of $\mathbf{A}$. Suppose E_B induces a spanning tree of G , then it suffices to prove that $\tilde{\mathbf{A}}_B$ is invertible. We shall prove this by showing that the rows of $\tilde{\mathbf{A}}$ are linearly independent. Suppose

$\mathbf{w} \in \mathbb{R}^{n-1}$ is such that $\mathbf{w}^T \tilde{\mathbf{A}}_B = \mathbf{0}$, then $\mathbf{w}^T \tilde{\mathbf{A}}_i = 0$ for all $i \in B$. Let $e_i = (v_k, v_h)$. Observe that if $v_n \notin e_i$, then $w_k = w_h$. If $v_k = v_n$, then $w_h = 0$ and vice versa. Since E_B induces a spanning tree T , there is some $j \in B$ such that e_j is incident to v_n , and so there is some $w_r = 0$. Note that for all neighbours v_m connected to v_r in T , $w_m = 0$. This means $\mathbf{w} = \mathbf{0}$. □

Now, we consider the reduced cost. Recall that $\mathbf{p}$ such that $\mathbf{p}^T = \mathbf{c}_B^T \tilde{\mathbf{A}}_B^{-1}$ is an optimal solution to the dual problem. Therefore, the reduced costs are

$$\bar{\mathbf{c}}^T = \mathbf{c}^T - \mathbf{p}^T \tilde{\mathbf{A}}.$$

Since the n -th row of $\mathbf{A}$ is removed in the truncated matrix, we have

$$\bar{c}_{(i,j)} = \begin{cases} c_{(i,j)} - (p_i - p_j) & \text{if } i, j \neq n \\ c_{(i,j)} - p_i & \text{if } j = n \\ c_{(i,j)} + p_j & \text{if } i = n \end{cases}.$$

By defining $p_n = 0$, we obtain $\bar{c}_{(i,j)} = c_{(i,j)} - (p_i - p_j)$ for all $(i, j) \in E(G)$. Let B be a basis inducing a spanning tree T on the network G . Note that if $(i, j) \in B$, we must have $\bar{c}_{(i,j)} = 0$, so we have

$$\begin{aligned} p_i - p_j &= c_{(i,j)} \quad \text{for all } (v_i, v_j) \in E(T), \\ p_n &= 0. \end{aligned}$$

Note that $V(T) = V(G)$, so solving the above system of equations gives the values of the p_i 's for all $i = 1, 2, \dots, n$, which then allows us to compute $\bar{\mathbf{c}}$.

Suppose we take a basis B corresponding to the spanning tree T of a network G . If there is some $(i, j) \notin B$ such that $\bar{c}_{(i,j)} < 0$, it means the edge (v_i, v_j) should be added to improve the flow. Note that there is an undirected v_i - v_j path in T , so $T + (v_i, v_j)$ contains a cycle C containing v_i and v_j . We define the direction of (v_i, v_j) as the forward direction and let C_f and C_b be the sets of forward and backward edges in C respectively.

To improve the cost, we need to increase the flow in (v_i, v_j) as much as possible. Suppose we set $x_{(i,j)} = \theta^*$, then the amount of forward flow in every $C_f \in C_f$ increases by θ^* , which means we need to reduce the backward flow in every $e_b \in C_b$ by θ^* as well. Intuitively, we can take

$$\theta^* = \min_{(v_k, v_\ell) \in C_b} x_{k\ell}.$$

In particular, if $C_b = \emptyset$, $\theta^* = \infty$ and the optimal cost is $-\infty$. Otherwise, if $x_{pq} = \theta^*$, we will choose (p, q) to leave the basis.

Technique 4.2.4 ► Network Simplex Method

Consider an uncapacitated minimum cost flow problem on a network G with node-arc incidence matrix $\mathbf{A}$. The network simplex method finds the optimal flow vector $\mathbf{x}^*$ by the following procedures:

1. Take a spanning tree $T_0 \subseteq G$ and find a basis B from $E(T_0)$ and a feasible tree solution $\mathbf{x}_0$.
2. At the k -th iteration, compute the dual vector $\mathbf{p}_k^T = \mathbf{c}_{B_k}^T \tilde{\mathbf{A}}_{B_k}^{-1}$. Define $p_{n_k} = 0$.
3. For each edge $u_i u_j \in E(G)$, compute the reduced cost $\bar{c}_{(i,j)} = c_{(i,j)} - (p_{i_k} - p_{j_k})$.
4. If $\bar{c}_{(i,j)} \geq 0$ for all $(i, j) \in E(G)$, then $\mathbf{x}_k$ is optimal.
5. Otherwise, choose $(i, j) \notin B$ with $\bar{c}_{(i,j)} < 0$.
6. Let C_k be the cycle contained in $T + u_i u_j$ and C_f and C_b be the forward and backward edges in C_k respectively, where $u_i u_j$ induces the forward direction.
 - (a) If $C_b = \emptyset$, then the optimal cost is unbounded.
 - (b) Otherwise, compute

$$\theta^* := x_{pq} = \min_{(u_k, u_\ell) \in C_b} x_{k\ell}.$$

7. Update $\mathbf{x}_k$ to $\mathbf{x}_{k+1}$ by

$$\widehat{x_{k\ell}} = \begin{cases} x_{k\ell} + \theta^* & \text{if } u_k u_\ell \in C_f \\ x_{k\ell} - \theta^* & \text{if } u_k u_\ell \in C_b \\ x_{k\ell} & \text{otherwise} \end{cases}.$$

8. Update $T_{k+1} = (T - u_p u_q) \cup u_i u_j$.

Similar to Technique 2.4.1, we can also deploy a two-phase method to initialise a feasible tree solution.

Technique 4.2.5 ► Network Two-Phase Method

Consider an uncapacitated minimum cost flow problem on an n -vertex network G with node-arc incidence matrix $\mathbf{A}$ and supply vector $\mathbf{b}$. The network two-phase method finds the optimal flow vector $\mathbf{x}^*$ by the following procedures:

1. Construct an auxiliary network G' on $V(G)$ with

$$E(G') := E(G) \cup \{u_i u_n \notin E(G) : b_i \geq 0\} \cup \{u_n u_j \notin E(G) : b_j < 0\}.$$

2. Initialise flow costs on G' by

$$c'_{(i,j)} = \begin{cases} 0 & \text{if } u_i u_j \in E(G) \\ 1 & \text{otherwise} \end{cases}.$$

3. Take $B := \{u_i u_n \in E(G') : b_i \geq 0\} \cup \{u_n u_j \in E(G') : b_j < 0\}$ as an initial basis. Run network simplex method on G' until a basis B_0 containing no edge from $G' - G$ is found.
4. Take B_0 as an initial basis to run network simplex method on G .

4.3 Integrality

Network problems exhibit an interesting feature that most quantities and parameters of interest are integers if the network data is given in integers.

Theorem 4.3.1 ► Integrality of Network Optimisation Problems

Let G be an n -vertex network with a connected underlying graph, then $\tilde{\mathbf{A}}_B^{-1}$ has only integer entries for any basis B .

Proof. For each vertex $v_i \in V(G)$ where $1 \leq i \leq n-1$, Consider a network flow problem (P_i) defined on a network $H_i \subseteq G$ with $V(H_i) = V(G)$ and $E(H_i) = B$, such that $b_i = 1$ and $b_n = -1$, while all other nodes are transshipment nodes with no external supply nor demand. Clearly, the flow balance equation for (P_i) is

$$\mathbf{A}_B \mathbf{x}_B = \mathbf{b}, \quad \text{where } \mathbf{b} = \mathbf{e}_i - \mathbf{e}_n.$$

Truncating the equation by taking the first $(n-1)$ rows yields $\tilde{\mathbf{A}}_B \mathbf{x}_B = \mathbf{e}_i$. Therefore, we have $\tilde{\mathbf{A}}_B^{-1} \mathbf{e}_i = \mathbf{x}_B$. By Proposition 4.2.3, H_i is a spanning tree of G and so there is a unique path between any pair of vertices. Therefore, the flow is either 1 or -1 depending on the directions of the edges. This implies that $\tilde{\mathbf{A}}_B^{-1} \mathbf{e}_i$ has only integer entries, and so $\tilde{\mathbf{A}}_B^{-1}$ is an integer matrix. $\square$

From Theorem 4.3.1, we see that if the supply vector $\mathbf{b}$ is also integer, then every basic solution $\mathbf{x}$ must be integer since $\mathbf{x}_B = \tilde{\mathbf{A}}_B^{-1} \mathbf{b}$ is now integer. Similarly, the dual basic solution $\mathbf{p} = \mathbf{c}_B^T \tilde{\mathbf{A}}_B^{-1}$ is also integer if the cost vector $\mathbf{c}$ is integer. This leads to the following easy corollary:

Corollary 4.3.2 ► Integrality of Optimal Solution in Network Flow Problems

Let G be an n -vertex network with a connected underlying graph. Consider a network flow problem (P) such that an optimal solution exists. If the supply vector $\mathbf{b}$ consists of purely integer entries, then there is an integer optimal solution; if the cost vector $\mathbf{c}$ consists of purely integer entries, then there is an integer dual optimal solution.